FINAL PROJECT REPORT

VeriDoc

Insurance Document Authenticity, Fraud Detection and Trusted-Source Verification

A decision-support platform that measures documentary evidence and hands the decision to a named human investigator

◆

Author	**REDOANUZZAMAN**
Programme	**⟨programme / course code⟩**
Institution	**⟨institution / department⟩**
Supervisor	**⟨supervisor⟩**
Platform version	**Platform 1.4.0 · rules 2026.09.1**
Date of submission	**18 September 2026**

The deployed system referenced throughout this report is available at https://veridoc.rmaikr.top

Abstract

Insurance claims are settled on the strength of documents — hospital invoices, prescriptions, identity papers, repair estimates and payment advices — that an insurer receives as files rather than as originals. A claims handler cannot inspect the paper, cannot see the software that produced the file, and has neither the time nor the instruments to compare a bundle of documents field by field against each other and against the claim record. This report presents **VeriDoc**, a decision-support platform for insurance document examination that closes that gap without replacing the examiner.

VeriDoc ingests a claim document and runs it through a fourteen-stage analysis pipeline that combines six independent families of evidence: structural and metadata forensics on the file itself, pixel-level image forensics, typography analysis, schema-driven optical character recognition and field extraction, twenty deterministic internal-consistency rules, field-by-field cross-document comparison across the claim bundle, a synthetic-media (AI-generated content) indicator, and lookups against trusted external registries. Each signal is recorded with the value measured, a calibrated confidence, the region of the page it refers to, and the detector and model version that produced it. A transparent, saturating risk model combines the signals into a 0–100 triage ranking and one of three bands — Low Concern, Review Recommended, High Concern.

The implemented system comprises 18,725 lines of Python across 110 modules and 11,077 lines of TypeScript across 81 modules, over a 25-table PostgreSQL schema. It is verified by 174 automated tests, all passing, which execute against a real PostgreSQL database and real object storage rather than against mocks, and by a scripted walkthrough that exercises all twenty product requirements against a live instance. The platform is deployed and running in production behind a reverse proxy with TLS.

The central design commitment, and the contribution this report argues for, is *epistemic restraint*. No code path in the platform can output a verdict that a document is genuine or fraudulent. Every detector's limitations and benign explanations are documented in the product itself, not only in this report; deterministic rules report the checks they passed as well as those they failed; a bundled demonstration provider can never be presented as a real external confirmation; and the server refuses to let a claim be approved while a high-severity finding remains unadjudicated. The machine measures; the human decides, in writing, on the record.

Keywords

document forensics; insurance fraud detection; optical character recognition; information extraction; explainable decision support; human-in-the-loop systems; audit trail integrity; accessible data visualisation.

Table of Contents

Abstract **2**
Table of Contents **3**
List of Figures **3**
List of Tables **3**
1 Introduction **7**
1.1 Background 7
1.2 Problem statement 7
1.3 Why decision support rather than decision making 8
1.4 Structure of this report 8
2 Aim, Objectives and Scope **9**
2.1 Aim 9
2.2 Objectives 9
2.3 Scope and exclusions 10
2.4 Deliverables 11
3 Review of Existing Approaches **12**
3.1 Manual examination by a claims handler 12
3.2 Rule-only claim management systems 12
3.3 Statistical fraud scoring on claim metadata 12
3.4 Standalone image-forensics research tools 13
3.5 Commercial document-verification services 13
3.6 Gap analysis 13
4 Methodology **15**
4.1 Development approach 15
4.2 Design principles 15
4.3 The provider-interface strategy 16
4.4 A generated, deterministic evaluation corpus 16
4.5 Technology selection 17
5 System Design and Architecture **19**
5.1 Overall architecture 19
5.2 Component decomposition 19
5.3 Data model 20
5.4 The analysis pipeline 21
5.5 The risk model 23
5.6 Security architecture 25
5.7 Console architecture 26
5.8 Deployment architecture 27
6 Implementation **28**
6.1 Ingest and validation 28
6.2 Rendering, OCR and field extraction 28
6.3 Forensic detectors 29
6.4 Tampering correlation 31

6.5 The consistency rule engine....31
6.6 Cross-document comparison....32
6.7 Synthetic-media analysis....33
6.8 Trusted-source verification....33
6.9 The risk engine....34
6.10 Evidence reporting....34
6.11 The investigator console....35
6.12 Interface revision....39
7 Testing and Validation....41
7.1 Test strategy....41
7.2 Automated test suite....41
7.3 Requirement walkthrough....43
7.4 Accessibility validation....43
7.5 Static analysis and build verification....43
8 Results and Evaluation....44
8.1 The evaluation corpus....44
8.2 Worked example....44
8.3 Behaviour on clean documents....45
8.4 Evaluation against the objectives....46
8.5 Operational characteristics....47
9 Limitations, Ethical and Legal Considerations....48
9.1 What the platform cannot establish....48
9.2 Detector-level limitations....48
9.3 Demonstration providers....49
9.4 Personal and special-category data....49
9.5 Automation bias and the human in the loop....49
9.6 Accessibility....50
10 Conclusion and Future Work....51
10.1 Conclusion....51
10.2 Future work....51
11 References....53
Appendix A Detector Catalogue....55
Appendix B Risk Weights and Ceilings....60
Appendix C API Endpoint Reference....62
Appendix D Test Inventory....65
Appendix E Data Dictionary....67
Appendix F Configuration Reference....69
Appendix G Deployment Runbook....71
Appendix H Console Screens....73

List of Figures

Figure 1 The operations dashboard. Counters are operational rather than decorative; every chart can be switched to the table of rows behind it. 35

Figure 2 The document queue. The risk band is always shown with its name and its score, never as a colour alone. 36

Figure 3 The investigation workstation on a document with an edited total. The printed total of $8,500.00 is boxed on the page; the subtotal of its own line items is $6,500.00; the issuer's record and the claim form both state 6,500.00. 37

Figure 4 The consistency panel. Passed and not-applicable checks are displayed alongside failures, so a clean result is evidence rather than an absence. 38

Figure 5 The detection-rules screen. Thresholds are configuration held in the database, not constants in code, and every change is audited. 39

Figure 6 The sign-in screen as deployed. No password is displayed or pre-filled; the demonstration panel lists role names and addresses only. 40

Figure 7 The claims register. A claim is the unit of investigation: its documents are compared against each other and against the claim particulars. 73

Figure 8 Open investigations with the per-investigator workload panel. A high-concern claim opens a case automatically, assigned to the least-loaded investigator. 74

Figure 9 Trusted-source verification: configured providers with their mode, and the request history with outcome filters. Demonstration providers are badged. 74

Figure 10 The forensics panel, showing findings, the full metadata table with anomalous rows annotated, and the raw measurements — including metrics that did not fire. 75

Figure 11 System settings. Band thresholds, retention, review policy and privacy policy are configuration held in the database, editable by an administrator. 75

Figure 12 Provider configuration. Each provider lists the *names* of the environment variables it requires; credential values are never stored in or displayed by the platform. 76

Figure 13 The audit trail with the hash chain verified at the top of the page, showing the entry and previous hashes and the before-and-after field differences. 76

Figure 14 The report register. Each report records its own SHA-256 and the SHA-256 of the document it describes, together with the pipeline and rules versions in force. 77

List of Tables

Table 1 Project objectives and the sections in which each is addressed. 9

Table 2 Project deliverables as submitted. 11

Table 3 Capability comparison. ● full, ◖ partial, ○ absent. 13

Table 4 Design principles and the mechanisms that enforce them. 15

Table 5 Principal technology choices and their justification. 17

Table 6 Principal backend packages and their responsibilities. 19

Table 7 Schema groups. The full data dictionary is Appendix E. 20

Table 8 The fourteen pipeline stages in execution order. 21

Table 9 Per-category ceilings, and the reasoning for their ordering. 24

Table 10 Threats and the controls that address them. 25

Table 11 Role capabilities. The full permission matrix is enforced server-side on every endpoint. 26

Table 12 Production services, networks and exposure....27
Table 13 The twenty consistency rules by family. Severities and weights are the seeded defaults....31
Table 14 Pairing policies. A comparison outside its policy is recorded as skipped, with the reason....32
Table 15 Verification outcome grading....33
Table 16 Test suite composition and coverage areas. Appendix D lists the assertions per file....41
Table 17 Evaluation scenarios and the evidence the platform recovers from each....44
Table 18 Principal risk contributions for DOC-2026-000004, as returned by the risk engine....45
Table 19 Objectives against outcomes....46
Table 20 Measured operational characteristics of the submitted system....47
Table 21 Signals most frequently produced by benign causes....48
Table 22 PDF metadata findings....55
Table 23 Image metadata findings....55
Table 24 Pixel-suite findings and their gating thresholds....56
Table 25 PDF structural findings....56
Table 26 Typography findings....57
Table 27 Source reliabilities used in the noisy-OR combination. The first three are high-reliability sources that may raise a region alone....58
Table 28 Synthetic-media indicators, their measurement and their published weights....58
Table 29 Likelihood bands. There is no code path that can emit a verdict....58
Table 30 Security findings....59
Table 31 Category ceilings, rule counts and the fallback weight applied when a finding code has no row of its own....60
Table 32 Highest and lowest seeded weights, illustrating the model's ordering of evidence....60
Table 33 System and authentication endpoints....62
Table 34 Claim, document and dashboard endpoints....62
Table 35 Verification, investigation, report, audit, user and administration endpoints....63
Table 36 Notable assertions per test file, emphasising the negative and wording assertions....65
Table 37 Table reference. Key columns only; every table also carries its identifier and timestamps...67
Table 38 Principal configuration variables....69
Table 39 Routine operations....71
Table 40 Expected failure modes and the system's behaviour....72

1 Introduction

1.1 Background

An insurance claim is, in practice, a bundle of documents. A health claim arrives as a hospital invoice, a discharge summary, a prescription, a copy of an identity document and a completed claim form; a motor claim as a repair estimate, a vehicle registration document, a police report and a payment advice. The insurer's decision to pay, to decline or to investigate rests almost entirely on whether those documents are what they purport to be and whether they agree with each other and with the policy record.

Two changes have made that judgement materially harder. First, submission is now digital: the insurer receives a PDF or a photograph, not paper. The physical cues an experienced examiner relies on — paper stock, ink, an embossed seal, a wet signature — are simply absent. Second, the tools needed to alter a document convincingly are ubiquitous and free. A total can be re-typed in a PDF editor, a date changed in a photograph of a bill, a template downloaded and completed with plausible but fictitious particulars. Generative image models add a further category: a document that never existed in any form.

What a digital submission does provide, in compensation, is *measurable structure*. A PDF records the software that produced it, the order in which it was created and last saved, whether revisions were appended incrementally, which font resource each character was drawn from, and what raster images are embedded in it. A photograph carries camera metadata, a compression history, and a sensor-noise signature that is not uniform across a composited image. None of these is visible to a human reader, and none is conclusive on its own — but each is a real, repeatable measurement of the file in hand.

1.2 Problem statement

The problem this project addresses is therefore not "can a machine decide whether this document is fake". It is the narrower, tractable and considerably more useful problem:

> **Problem**
>
> Given a claim document and the claim it belongs to, produce a complete, explainable and auditable record of every measurable property that bears on the document's authenticity and its agreement with the rest of the claim — ranked so that a human investigator's limited attention is spent where it is most likely to matter, and presented so that the investigator can verify each signal against the document itself rather than trusting the system's summary.

Four properties follow from that statement and constrain everything in the design. The output must be **explainable** — an investigator must be able to see which pixels, which extracted value or which external record produced a signal. It must be **calibrated and bounded** — a signal must state its own uncertainty and its benign explanations. It must be **auditable** — who saw what, who decided

what and why must survive scrutiny after the fact. And the system must be **subordinate to the human** — no automated output may become a decision by default.

1.3 Why decision support rather than decision making

It is technically possible to train a classifier that emits a fraud probability for a document, and commercially tempting to do so. This project deliberately does not. Three reasons, in descending order of importance:

- **The inference is not available in the evidence.** Every signal the platform can measure has innocent explanations that occur frequently in legitimate claim traffic. A hospital that crops a scan in an image editor, a billing system that appends a digital signature as an incremental PDF revision, a claimant who re-saves a photo to compress it before uploading — each produces exactly the measurement that a manipulated document produces. A probability presented on top of that ambiguity does not resolve it; it conceals it.
- **The consequences are borne by a person.** A declined health claim is a medical bill an individual must now pay. A system whose output is a number that a handler is institutionally encouraged to accept converts a measurement into an accusation without anyone having examined the document.
- **A verdict destroys the audit trail.** If the output is "fraudulent, 0.91", there is nothing to review. If the output is "the printed total does not match the sum of its own line items, and the issuer's record states a different amount", the reviewer can check both claims, and a regulator can later check the reviewer.

The platform's risk score is therefore defined and described throughout the product as a **triage ranking** — how much human attention this document should receive — and the interpretation note stating that it is not a probability of fraud is emitted with every assessment by the risk engine itself, not merely written into documentation.

1.4 Structure of this report

Section 2 states the aim, objectives and scope. Section 3 reviews the approaches currently available and identifies the gap. Section 4 sets out the methodology and the design principles that govern the implementation. Section 5 presents the system design: architecture, data model, pipeline, risk model and security. Section 6 documents the implementation of each subsystem. Section 7 describes testing and validation, and Section 8 the results obtained against the evaluation corpus. Section 9 states the limitations and the ethical and legal considerations, and Section 10 concludes. Eight appendices carry the full detector catalogue, the risk weights, the API surface, the test inventory, the data dictionary, the configuration reference, the deployment runbook and the console screens.

2 Aim, Objectives and Scope

2.1 Aim

To design, implement, test and deploy a working platform that examines insurance claim documents for authenticity and internal and external consistency, presents every finding with its evidence and its uncertainty, ranks documents for human attention, and records the human decision that follows — such that the platform is genuinely useful to an investigator without ever substituting for one.

2.2 Objectives

The aim decomposes into eleven objectives, each of which is evaluated in Section 8.4.

Table 1. Project objectives and the sections in which each is addressed.

#	Objective	Addressed in
O1	Accept PDF and raster claim documents through a validated, hash-identified ingest path that never loses or mutates the original.	5.6, 6.1
O2	Recover the text and the structured fields of a document, with a page coordinate for every value so that a claim can be pointed at.	6.2
O3	Measure the structural, metadata, pixel-level and typographic properties of the file that bear on authenticity.	6.3
O4	Correlate independent forensic signals into region-level tampering indications, rather than reporting each noisy metric separately.	6.4
O5	Apply deterministic internal-consistency rules to the extracted fields and report passes as well as failures.	6.5
O6	Compare documents field by field across a claim bundle and against the claim record, without manufacturing findings from legitimate differences.	6.6
O7	Provide a synthetic-media indicator behind a provider interface, labelled so that a heuristic can never be mistaken for a trained model.	6.7

#	Objective	Addressed in
O8	Verify document particulars against trusted external registries behind one interface, grading contradiction differently from missing detail.	6.8
O9	Combine all signals into a transparent, configurable, bounded triage score whose every term is published with the result.	5.5, 6.9
O10	Present the evidence to an investigator such that each finding can be verified against the document, adjudicated with a written reason, and folded back into the score.	6.11
O11	Make the whole process auditable, role-separated and deployable, with tamper-evident records of every action.	5.6, 5.8, 6.13

2.3 Scope and exclusions

In scope. The following are implemented and operating in the deployed system:

- Eleven document types with declared extraction schemas plus an `other` fallback: medical invoice, hospital bill, prescription, identity document, insurance certificate, claim form, vehicle document, repair estimate, receipt, police report and bank payment document.
- PDF, JPEG, PNG and TIFF inputs up to 25 MB and 60 pages.
- English-language recognition and extraction.
- Four user roles with a 33-permission matrix enforced server-side on every endpoint.
- A complete investigator console, a PDF evidence report, and a hash-chained audit trail with an integrity-verification endpoint.

Explicitly out of scope. Each exclusion is a deliberate decision, not an omission:

- **A fraud verdict or a fraud probability.** See Section 1.3.
- **A trained synthetic-media classifier.** The bundled provider is a deterministic eight-feature heuristic. Training a credible detector requires a labelled corpus of document images that this project does not have, and shipping an under-evaluated model behind a confident-looking number would be worse than shipping a declared heuristic. The production seam for a real model is implemented and tested.
- **Live connections to real insurer, hospital, government or banking registries.** These require commercial agreements and accredited access. The interface, the outcome grading, the payload construction and the production HTTP provider are all implemented; the bundled providers answer from a sample registry and are labelled as demonstration data at every layer.
- **Non-Latin scripts and non-English documents**, which would need different OCR models, different label synonym sets and different date and money conventions.
- **Handwriting recognition** and signature biometrics.

- **Claim-level behavioural fraud analytics** — provider billing patterns, claimant history, network analysis. These operate on claim metadata rather than on the document, and are a complementary rather than a competing capability (Section 3.3).

2.4 Deliverables

Table 2. Project deliverables as submitted.

Deliverable	Detail
Analysis service	FastAPI application, 110 Python modules, 18,725 lines; 14-stage pipeline; 58 HTTP endpoints across 12 routers.
Investigator console	Next.js 15 application, 81 TypeScript modules, 11,077 lines; 12 authenticated screens.
Database schema	25 PostgreSQL tables under Alembic migration control.
Detection content	99 configured detection rules (20 consistency rules plus 79 detector and verification weights), 17 provider configurations, 12 extraction schemas, 60 field specifications.
Test suite	174 automated tests against real PostgreSQL and real storage; a 23-check scripted requirement walkthrough.
Documentation	Seven technical documents in the repository (architecture, detection catalogue, API, deployment, dokploy, security, walkthrough) plus this report.
Deployment	Self-managed Docker Compose stack behind Traefik with Let's Encrypt TLS, live at https://veridoc.rmaikr.top.

3 Review of Existing Approaches

Five approaches are currently available to an insurer that wants to know whether a claim document can be relied upon. Each is reviewed below on the same four criteria: what it actually measures, whether its output is explainable to a reviewer, whether it produces an auditable record, and where it fails.

3.1 Manual examination by a claims handler

The default. A handler reads the documents, checks the arithmetic if there is time, and forms a judgement. Its strength is contextual: an experienced handler recognises that a particular clinic's invoices always look a certain way, and notices when one does not.

Its limits are structural rather than a matter of diligence. The handler cannot read PDF object structure, cannot compare a font resource against its neighbours, and cannot measure a compression history. Cross-document comparison across a six-document bundle involves dozens of field pairs and is performed, at volume, by sampling rather than exhaustively. And the record of the examination is typically a note, not a reproducible trace — so a decision cannot be re-examined on the same evidence a year later.

3.2 Rule-only claim management systems

Most claim platforms embed configurable business rules: an amount over a threshold, a treatment date outside the cover window, a provider not on the panel. These are deterministic, cheap, explainable and entirely appropriate — VeriDoc implements twenty of them itself (Section 6.5).

The limitation is the input, not the mechanism. Such rules operate on fields that were keyed in or supplied by an intermediary, not on fields *read from the document*. A rule engine given a total of 6,500 cannot detect that the document says 8,500, because it never saw the document. Rules also cannot reach properties that are not fields at all: no rule expresses "this number was typed in a different font from the rest of its line".

3.3 Statistical fraud scoring on claim metadata

Established practice in the industry, and genuinely effective: a model over claim, claimant, provider and network features — claim frequency, provider billing profile, time from policy inception to first claim, geographic clustering — produces a prioritisation score for a claim. Mature deployments detect organised and repeat fraud that no document examination could reach, because the signal lives in the pattern across claims rather than in any one file.

Three limitations are relevant here. Such models are **document-blind**: a perfectly edited invoice from a first-time claimant with an unremarkable profile scores as unremarkable. They are frequently **not explainable** at the level a reviewer needs — a gradient-boosted score over 200 features cannot be checked against the document, only accepted or ignored. And they **encode historical decisions**: trained on past investigation outcomes, they reproduce whatever selection

bias those investigations had. This approach and the present one are complementary, and Section 10.2 proposes their combination rather than a choice between them.

3.4 Standalone image-forensics research tools

A substantial body of academic work provides the measurements this project relies on: error-level analysis (Krawetz, 2007), copy-move detection by keypoint matching (Fridrich et al., 2003; Amerini et al., 2011), sensor-noise inconsistency (Lukáš et al., 2006), resampling traces (Popescu and Farid, 2005), and the broader methodology of Farid (2009, 2016). Research implementations of each exist and are often openly available.

Three problems arise in applying them to claim documents specifically. First, **the corpus is wrong**: these detectors were developed and calibrated on natural photographs, whereas a claim document is mostly blank paper carrying high-contrast printed text. Section 6.3 documents two detectors — resampling periodicity and JPEG lattice energy — which fire on essentially every printed page because print has exactly the regular structure they look for; this project measures both, records them as context, and explicitly declines to raise them as signals on text-heavy rasters. Second, **they are instruments, not systems**: they output a heatmap for an analyst to interpret, with no extraction, no claim context, no adjudication and no audit record. Third, **they do not cover digital-origin documents at all** — the dominant case in claims is a PDF that was never a photograph, where the signal lives in object structure and font resources rather than in pixels.

3.5 Commercial document-verification services

A market exists for identity-document verification and, more recently, for general document-fraud detection offered as an API. These are effective within their design envelope — identity documents from known issuers, with known security features, are a well-bounded problem.

From the perspective of an insurer examining a mixed bundle, the difficulties are that coverage is narrow (an identity verifier has nothing to say about a hospital invoice), that the output is characteristically a **score without evidence**, which cannot be adjudicated or defended, that sending medical documents to a third-party API raises data-protection questions that are not always answerable, and that cross-document consistency within one claim — a property of the *bundle*, not of any file — is outside the service's view entirely.

3.6 Gap analysis

Table 3. Capability comparison. ● full, ◖ partial, ○ absent.

Capability	Manual	Rule engine	Metadata ML	Forensics tool	Doc-verify API	VeriDoc
Reads the document itself	●	○	○	◖	●	●
PDF structure and metadata analysis	○	○	○	◖	◖	●
Pixel-level forensics	○	○	○	●	◖	●

Capability	Manual	Rule engine	Metadata ML	Forensics tool	Doc-verify API	VeriDoc
Typography / per-character font analysis	○	○	○	○	○	●
Field extraction with page coordinates	●	○	○	○	◖	●
Deterministic consistency rules	◖	●	○	○	◖	●
Cross-document agreement in a bundle	◖	○	○	○	○	●
External registry verification	◖	◖	○	○	◖	●
Synthetic-media indication	○	○	○	◖	◖	◖
Behavioural / network fraud signal	◖	○	●	○	○	○
Explainable to the level of a page region	●	●	○	◖	○	●
States detector limitations in the output	n/a	○	○	○	○	●
Mandatory human adjudication with reasons	●	○	○	○	○	●
Tamper-evident audit trail	○	◖	◖	○	◖	●

The gap is not a missing detector. Every individual measurement VeriDoc performs has precedent in the literature or in a commercial product. The gap is that **no available system assembles document-level measurement, field-level extraction, claim-level comparison and external verification into one explainable evidence record, ranks it for human attention, and then makes the human adjudication a first-class, audited part of the record**. That assembly — and the epistemic discipline it requires — is what this project contributes.

4 Methodology

4.1 Development approach

Development was iterative and vertical-slice first. Rather than building all detectors and then a user interface, the first milestone was a single document travelling the entire path — upload, validation, render, OCR, extraction, one rule, one forensic measurement, a score, a screen — because the interfaces between those stages are where the design risk lives. Detector families were then added one at a time, each with tests, and each with a documented statement of what it cannot tell you written at the same time as the code.

Two practices were maintained throughout. **The evidence contract came before the detector.** Every finding, regardless of family, carries the same fields — code, category, severity, confidence, page, region, description, evidence blob, detector identity and model version — which is why a single console component can render a deterministic rule failure and a model inference without special-casing either. **Demo data was generated, not collected** (Section 4.4), so that every anomaly the platform reports on the sample corpus is a real measurement of a real file.

4.2 Design principles

Six principles were adopted at the outset and are traceable to specific mechanisms in the implementation. They are stated here because most of the non-obvious decisions in Sections 5 and 6 follow from them.

Table 4. Design principles and the mechanisms that enforce them.

Principle	Enforcing mechanism
P1. Measure; do not conclude. No output asserts authenticity or fraud.	The risk engine emits a fixed interpretation note with every assessment; the synthetic-media provider returns a band, never a verdict; risk-band wording is centralised in one module so that "High Concern" cannot drift into "fraudulent" anywhere in the product.
P2. State the limits in the product. A detector whose limitations are undocumented will be over-trusted.	Every finding carries an uncertainty note and its benign explanations; the forensics panel displays the raw measurements *including metrics that did not fire*; a degraded pipeline stage is disclosed in the assessment so that an absence of findings is not read as evidence of integrity.
P3. Never lose the original. Evidence must outlive any processing error.	Originals are written once, fsynced, atomically renamed and chmod `0o440` in a storage scope separate from derived artifacts; no stage mutates or deletes them; a malware verdict quarantines without deleting; the retention job removes an original only if the document has *also* been soft-deleted.
P4. Human decisions survive the machine. Reprocessing must not discard judgement.	Confirm/dismiss decisions are snapshotted before findings are regenerated and re-applied to the equivalent new finding, keyed on kind, code, page and a 24-pixel-bucketed region.

Principle	Enforcing mechanism
P5. Configuration, not code. Thresholds an operator must tune must not require a deployment.	Rule severities, weights, parameters, category ceilings and band thresholds live in database tables read at run time and are editable by an administrator, with every change audited.
P6. Colour never carries meaning alone. The product is read for hours by people with varying colour vision.	Risk bands are always accompanied by their name; charts use a validated single-hue ordinal ramp rather than a green-to-red triad; every chart can be switched to its underlying table; the active navigation item is marked by a rail as well as a colour (Section 7.4).

4.3 The provider-interface strategy

Five capabilities in the pipeline depend on facilities that a real deployment would buy rather than build: optical character recognition, synthetic-media detection, trusted-source verification, object storage and malware scanning. Each is expressed as an abstract provider with at least one locally runnable implementation and a production seam behind the same interface.

The consequence is that the system is complete and testable with no external credentials at all, while a production provider can be substituted without touching the pipeline. Three properties were required of every provider interface:

1. **Failure is a first-class outcome.** A provider that cannot answer raises `ProviderUnavailable`, which degrades exactly one pipeline stage and is recorded; it never yields a fabricated result. The production HTTP synthetic-media provider raises rather than defaulting when the response lacks a numeric likelihood.
2. **Demonstration output is labelled at every layer.** A demo answer carries `is_demo_data`, a disclaimer naming the bundled registry, a `provider_mode` on the stored request, a demo badge in the console and an `[EXTERNAL — DEMO]` marker in the PDF report. There is no configuration in which demonstration output can be presented as a real external confirmation.
3. **Provider identity travels with the finding.** Provider key, model name and model version are stored on the finding and printed in the report, so a result can be attributed to the exact detector that produced it after the fact.

4.4 A generated, deterministic evaluation corpus

Evaluating a document-forensics system requires documents with known manipulations. Public corpora of manipulated *insurance* documents do not exist for good reasons, and hand-annotated real claim documents cannot be redistributed. Two unsatisfactory options remained: bundle a handful of files and fabricate the findings, or generate the corpus.

This project generates it. A specification describes each demonstration document, and a generator renders a real PDF or raster with the anomaly baked in *at the object or pixel level*: a genuine incremental PDF revision, a total drawn from a different font resource than the rest of its line, a value re-typed after capture with the resulting compression discontinuity, EXIF naming a raster

editor, an annotation overlaying the amounts block. The pipeline then has real evidence to find, and the demonstration exercises the production code paths rather than a fixture.

All randomness is seeded, so the same seed produces byte-identical documents, identical findings and identical scores on any machine — which is what makes the corpus usable as a regression test as well as a demonstration. The corpus comprises 10 claims and 22 documents covering nine scenario classes (Section 8.1).

> **A note on what this corpus can and cannot support**
>
> A generated corpus establishes that each detector fires on the property it claims to measure, and that the pipeline, scoring and review paths work end to end. It cannot establish a detection rate or a false-positive rate against real claim traffic, because the manipulations were authored by the same project that wrote the detectors. No such rates are claimed anywhere in this report. Establishing them requires a labelled corpus of genuine claim documents and is stated as future work in Section 10.2.

4.5 Technology selection

Table 5. Principal technology choices and their justification.

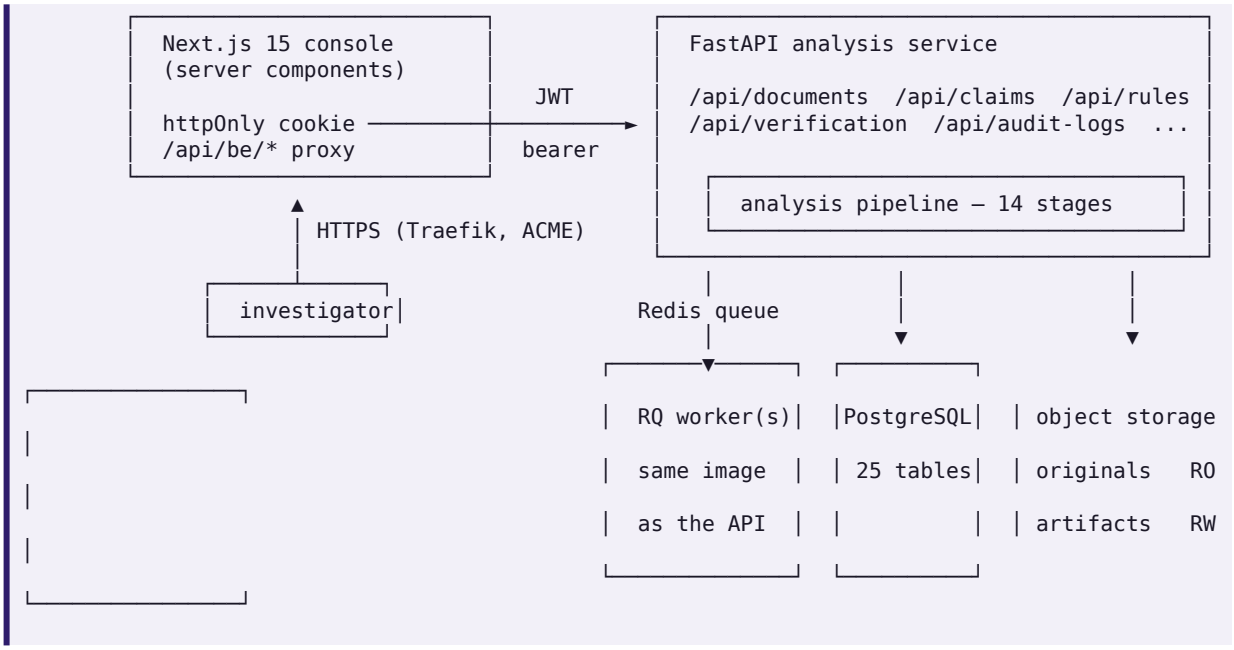
Concern	Choice	Justification
Analysis service	Python 3.12 / FastAPI 0.115	The document-analysis ecosystem — pypdf, pdfplumber, pikepdf, Pillow, OpenCV, NumPy, pytesseract — is Python-native. FastAPI supplies typed request validation and a generated OpenAPI schema without a separate specification to keep in step.
Persistence	PostgreSQL 16 / SQLAlchemy 2.0 / Alembic	Evidence is relational but findings are structurally heterogeneous; JSONB columns hold each detector's evidence blob without a schema migration per detector, while keys, constraints and indexes stay relational. A gap-free `IDENTITY` sequence supplies the audit chain's ordinal.
Queue	Redis 7 / RQ, with an in-process fallback	Analysis takes seconds to minutes and must not block an upload response. RQ is sufficient for a single queue and adds no broker to operate. When Redis is unreachable the API falls back to a background task and records which path ran, so the system remains usable on a laptop.

Concern	Choice	Justification
Console	Next.js 15 (App Router) / React 19 / TypeScript	Server components allow the access token to stay in an httpOnly cookie and be attached server-side, so no credential ever reaches the browser bundle — the decisive property for a console displaying medical documents.
Styling and charts	Tailwind CSS 3 / Radix primitives / Recharts	A token-driven design system makes the accessibility constraints of principle P6 enforceable centrally rather than per component; Radix supplies accessible behaviour for dialogs, menus and tabs.
Reporting	ReportLab	The evidence report must be a fixed-layout, self-contained artefact whose SHA-256 is recorded; generating it server-side in the same process that holds the evidence avoids a rendering service in the trust path.
Deployment	Docker Compose behind Traefik	Five services, one host, TLS terminated at the proxy with automatic certificate issuance. The API is published on no host port and carries no proxy route, so it is unreachable from the internet by construction rather than by firewall rule.

5 System Design and Architecture

5.1 Overall architecture

The system is four cooperating processes over two data stores. The console renders and proxies; the API validates, stores and orchestrates; the worker executes the pipeline; the datastores hold the evidence. The topology is reproduced below as deployed.



Two structural decisions in that diagram carry most of the security argument. **The API has no public route.** Only the console is routed by the reverse proxy; the API is reachable only on a private Docker network, and the console reaches it server-side. **Storage is split in two scopes.** `originals` holds the exact bytes the claimant uploaded, written read-only and never modified; `artifacts` holds derived data — page renders, error-level-analysis heatmaps, generated reports — which can be regenerated and purged independently. That split is what makes retention and secure deletion tractable without ever risking the evidence.

5.2 Component decomposition

Table 6. Principal backend packages and their responsibilities.

Package	Responsibility
`app/api/routes/`	Twelve routers — health, auth, dashboard, claims, documents, verification, investigations, reports, audit, users, admin — each endpoint declaring the permission it requires.
`app/core/`	Configuration, password hashing and JWT issue/verify, HMAC-signed object URLs, the RBAC matrix, the typed error hierarchy, the middleware stack and PII-aware logging.

Package	Responsibility
app/db/	Declarative models over 25 tables, shared mixins, the canonical enumerations, Alembic migrations, and the idempotent seeder including the demonstration-document generators.
app/services/pipeline.py	The fourteen-stage orchestrator: stage timing, status, provider snapshot and per-stage detail, with error isolation by stage.
app/services/{ocr,classify,extract}/	Text recovery, lexical-structural document classification, and schema-driven field extraction with page coordinates.
app/services/forensics/	PDF structure and metadata, embedded-raster extraction, the pixel suite, EXIF analysis, and typography including exact per-character font resources.
app/services/{aigc,tampering,consistency,crossdoc,verification}/	The synthetic-media provider interface, the region-level correlation layer, the twenty-rule consistency engine, claim-bundle comparison with pairing policies, and the six trusted-source providers.
app/services/risk/engine.py	The deterministic scoring model: weights, severity multipliers, per-category saturation, banding, explanation and coverage disclosure.
app/services/findings_store.py	The only module that knows the mapping across the five finding tables; supplies one uniform read model and carries human review decisions forward across reprocessing.
app/services/audit.py	Append-only audit trail with a SHA-256 hash chain, PII redaction before insertion, and chain verification.
app/services/{storage,malware,reports}/	Storage scopes with key validation, the static and ClamAV scanners, and the ten-section PDF evidence report.
app/workers/	Queue submission with an in-process fallback, the analysis and retention tasks, and the worker entry point.

5.3 Data model

The schema is 25 tables in nine groups. Its shape follows the evidence rather than the screens: a document's pages, recognition results, extracted fields, metadata entries, findings, verification requests and risk assessments are each first-class records, because each must be independently re-derivable, independently retained and independently cited in a report.

Table 7. Schema groups. The full data dictionary is Appendix E.

Group	Tables
Identity	roles, users
Claim	claims
Document	documents, document_pages, document_metadata, ocr_results, extracted_fields
Analysis	analysis_runs, risk_assessments
Findings	forensic_findings, aigc_findings, tampering_findings, consistency_findings, cross_document_findings

Group	Tables
Verification	`verification_requests`, `verification_results`
Human process	`investigations`, `investigation_notes`, `review_actions`
Output	`reports`
Governance	`audit_logs`, `detection_rules`, `provider_configs`, `system_settings`

Why findings occupy five tables

The five finding tables share one evidence contract through a common mixin — code, category, severity, confidence, page, region, description, evidence, detector, model version, review status and reviewer — yet they are stored separately. The reason is that their provenance and their retention differ in kind. A deterministic rule failure and a model inference are not the same sort of claim about the world, and treating them as rows in one table would erase that distinction precisely where it matters. Each table also carries the columns its family actually needs: a tampering finding has a tamper type and the extracted field its region covers; a synthetic-media finding has the provider, its demo flag and the per-indicator scores; a cross-document finding belongs to a *bundle* rather than to a single file and therefore carries a claim identifier, a counterpart document and both compared values.

The cost of that decision — five tables to query instead of one — is paid once, in `findings_store.py`, which is the only module aware of the mapping and which exposes a single uniform read model to the API, the risk engine and the report generator.

5.4 The analysis pipeline

Fourteen stages run in a fixed order through a context manager that records, for each: status, start and finish time, duration, provider snapshot, message, finding count and a stage-specific detail blob. Stages are classified *critical* or *non-critical*, which determines the outcome of the run as a whole.

Table 8. The fourteen pipeline stages in execution order.

#	Stage	Crit.	Function
1	File validation	yes	Re-reads the stored original and recomputes its SHA-256 against the ingest hash; records size, MIME and page count; reports byte-identical duplicates already on record.
2	Malware / security	yes	Static active-content scan or ClamAV. An infected verdict quarantines the document and halts stages 3–13 without deleting anything.

#	Stage	Crit.	Function
3	Page rendering	yes	Rasterises each page to PNG plus a thumbnail at 150 dpi, maximum edge 2400 px, into the artifacts scope.
4	OCR and text extraction	yes	Up to 30 pages through the active provider; persists words, boxes and confidences per page and records which route was taken.
5	Document classification	yes	Weighted lexical and structural scoring against per-type signatures; below the score floor the type is `other`.
6	Field extraction	yes	Applies the schema for the detected type: label synonyms, column geometry and value patterns, with a bounding box and a confidence for every value.
7	Metadata analysis	no	PDF document information and XMP, cross-reference generations, byte markers, fonts and embedded-image inventory; EXIF for rasters.
8	Image and PDF forensics	no	The pixel suite over embedded rasters or the image itself, PDF font analysis, and typography analysis per recognised page; writes forensic overlay layers.
9	Synthetic-media analysis	no	Per page, up to 12 pages, through the active provider; emits a band, never a verdict.
10	Tampering correlation	no	Fuses overlapping suspect regions from independent detectors and names the region by intersecting it with the extracted fields.
11	Consistency rules	no	Twenty deterministic rules over the extracted fields, with administrator overrides applied; records passes and non-applicable outcomes as well as failures.

#	Stage	Crit.	Function
12	Cross-document comparison	no	Compares this document against every sibling in the claim and against the claim record, under pairing policies.
13	Trusted-source verification	no	Runs the request plan for the document type against the configured providers; a missing reference skips the request with a stated reason.
14	Risk assessment	yes	Combines persisted findings and verification outcomes with the stage-coverage map into a score, a band, an explanation and the published formula.

Error isolation

The pipeline's central operational property is that failure is scoped to a stage. A provider that cannot answer yields `unavailable` and is retryable with the document untouched; any other exception yields `failed` with the exception type recorded; a stage whose inputs are absent yields `skipped` with a stated reason. A failed **critical** stage fails the run; a failed non-critical stage marks it `partial`. In either case the document's processing message states that the original is preserved and can be reprocessed, and the risk assessment records which stages were degraded — so that the absence of findings in those areas is disclosed as omission rather than read as evidence of integrity.

The second invariant is that human judgement survives reprocessing. Before findings are regenerated, confirm and dismiss decisions are snapshotted and then re-applied to the equivalent new finding, matched on kind, code, page and a region bucketed at 24 pixels. Re-running analysis therefore cannot silently discard a reviewer's work.

5.5 The risk model

The scoring model was required to be transparent, bounded, configurable and monotonic — and to be published with its own output, so that a reviewer can recompute it by hand. It has three steps.

```
        contribution(signal) = weight(code) × confidence × severity_multiplier

        category_score       = cap × (1 − exp(−1.3 × Σ contributions / cap))

        score                = round(100 × Σ category_scores / 72) −
mitigation
```

Severity multipliers are info 0.35, low 0.60, medium 1.00, high 1.35, critical 1.60. **Weights** are per finding code, stored in the `detection_rules` table and editable by an administrator, with a

per-category default when a code has no row. A **dismissed** finding is excluded and the exclusion is reported with the score; a **confirmed** finding has its confidence promoted to at least 0.95, because a human has established it.

The exponential form gives each category a **saturating** contribution towards a ceiling. This is the mechanism that stops a crowd of weak signals in one family from outweighing a single decisive signal in another, while still allowing a genuinely decisive family to reach High Concern on its own.

Table 9. Per-category ceilings, and the reasoning for their ordering.

Category	Ceiling	Reasoning
External verification	70	When the issuer's own record contradicts the document, that is the strongest evidence this platform can obtain.
Tampering indicators	58	Requires independent detectors to agree on one region, so it is more than any single metric.
Internal consistency	50	Deterministic, but conditional on extraction being correct.
Cross-document agreement	50	Deterministic, but conditional on both documents being extracted correctly.
File security	40	Decisive about the file, weaker about the claim.
Document forensics	34	Individually noisy measurements with frequent benign explanations.
Metadata provenance	24	Trivially strippable and routinely rewritten by legitimate tooling.
Synthetic-media indicators	20	Lowest, because the bundled provider is a heuristic rather than a calibrated model.

Bands are Low Concern below 40, Review Recommended 40–74 and High Concern 75 or above; the thresholds are system settings, validated as an increasing pair within 0–100. **Mitigation**: each corroborating external match credits 2 points, capped at 6 in total, and is suppressed entirely — with the suppression reported — whenever any high or critical signal is present. Corroboration may therefore lower a score that has no severe signal in it, and can never mask one that has.

Every assessment returns each signal with its weight, confidence and contribution, the per-category raw and saturated totals, the six principal contributors, a natural-language explanation, the coverage map, the weights version, the count of excluded dismissed signals, the formula as a string, and the fixed interpretation note. The score is reproducible from its own output.

5.6 Security architecture

The platform holds medical treatment details, diagnoses, identity-document numbers, dates of birth, addresses and bank details. The controls were designed around that content rather than added afterwards.

Table 10. Threats and the controls that address them.

Threat	Control
Stolen session token	The access token is held in an httpOnly, SameSite=Lax cookie and attached to API calls server-side, so it never enters the browser bundle; 45-minute access lifetime with a single transparent refresh; every action attributed in the audit trail.
Credential stuffing	Rate limits per IP *and* per account on sign-in; lockout for 15 minutes after six failures; identical error wording for an unknown account and a wrong password.
Privilege escalation through the interface	Authorisation is enforced server-side on every mutating endpoint; the console's permission matrix only hides controls.
Malicious upload	Content type sniffed from magic bytes before the declared type or the filename is trusted; structural parse; static active-content and polyglot checks; optional ClamAV; a suspicious or infected verdict quarantines *without* deleting.
Storage escape	Storage keys validated against a strict pattern and resolved paths checked against the scope root.
Document exfiltration by URL guessing	Nothing is public; bytes are served to an authenticated session or through an HMAC-signed link bound to the resource, the purpose and the user, default lifetime five minutes, with `Cache-Control: private, no-store`.
Tampering with the evidence record	Originals are write-once; findings carry detector and model version; the audit trail is append-only and hash-chained.
Insider rewriting history	`entry_hash = sha256(prev_hash ‖ canonical(entry))` over a gap-free database-assigned ordinal; a verification endpoint recomputes the chain and names the first divergence.
Over-trusting the machine	Reasons are mandatory on adjudication; claim approval is refused while a high or critical finding is unadjudicated; every probabilistic signal ships with its uncertainty and its benign explanations.
Leakage through logs	Document bodies are never logged; the log formatter redacts emails, long digit strings and card-like numbers; audit rows drop secrets and special-category fields before insertion.
Demo output mistaken for a real confirmation	`is_demo_data` on the result, a provider mode on the request, a badge in the console, an `[EXTERNAL — DEMO]` marker in the report and a disclaimer beside every answer.

Separation of duties

Four roles share 33 permissions, arranged so that no single role can both establish a finding and dispose of the claim it belongs to.

Table 11. Role capabilities. The full permission matrix is enforced server-side on every endpoint.

Role	Perms	May	May not
Administrator	33	Everything, including users, rules, providers, thresholds and the audit trail.	—
Fraud Investigator	24	Adjudicate findings, escalate, run verification, verify or reject a document, issue reports, read the audit trail.	Create or decide a claim; administer rules, providers or users.
Claims Officer	20	Create claims, upload documents, request information, record the claim decision, issue reports.	Confirm or dismiss a finding; read the audit trail.
Reviewer	15	Second-line confirm or dismiss of machine findings; add investigation notes.	Escalate, decide a claim, or run verification.

5.7 Console architecture

The console is a Next.js App Router application in which server components are the default. Each authenticated screen is an asynchronous server component that reads the httpOnly cookie, calls the API directly from the server, and streams HTML. Client components exist only where interaction demands them.

Browser-side mutations go through a same-origin proxy route rather than to the API directly. That route exists for three reasons, and the trade-off is deliberate: no token ever reaches the browser; an image tag for a page render or a forensic overlay works without credentials in a query string; and an expired access token is refreshed transparently, once, using the refresh cookie, so an investigator is never signed out mid-review. Hop-by-hop headers and inbound cookies are stripped in both directions, and a rotated token is written back as a fresh httpOnly cookie.

List state — filters, pagination, sort — lives in the URL and is read server-side, so a filtered queue is linkable and survives a reload. There is no client-side data-fetching or state-management library: a mutation posts, then refreshes the server component tree. Caching is disabled throughout, and document renders carry `Cache-Control: private, no-store`.

The investigation workstation

The workstation is the centre of the product and the design is split-screen by necessity: the document stays on the left while every panel on the right can draw a box around the pixels it is discussing. Two pieces of state are shared across the split — the active finding and the active extracted field — and they are mutually exclusive by construction. Clicking a finding, a risk contributor or an extracted field navigates the viewer to the right page and highlights the right

region; clicking a region on the page selects the corresponding finding card. Regions are positioned as percentages of the page's own pixel space inside a nested transform, so highlights stay correct under zoom, rotation and fit-to-width scaling.

5.8 Deployment architecture

The production deployment is a five-service Docker Compose project on a single host. Its security-relevant shape is that **only the console is routed**: the reverse proxy holds labels for the web service alone, and no service publishes a host port.

Table 12. Production services, networks and exposure.

Service	Image	Networks	Exposure
`postgres`	postgres:16-alpine, data checksums enabled	internal only	None. Health-gated by `pg_isready`.
`redis`	redis:7-alpine, append-only persistence	internal only	None. Health-gated by `redis-cli ping`.
`api`	built from `backend/`	internal only	**No proxy route and no host port.** Reached only as `http://api:8000` from the console's server side.
`worker`	the same image as `api`	internal only	None. Scalable with `--scale worker=n`; its health check is explicitly disabled because the image's baked-in probe targets the API port a worker does not serve.
`web`	built from `frontend/`, standalone Next output	internal + ingress	The only routed service: host rule on the application domain, TLS with automatic certificate issuance, and a permanent redirect from HTTP.

Both images run as unprivileged users. The backend image serves both the API and the worker — the command differs, the code does not — and installs the pipeline's system dependencies explicitly: Tesseract for pixel recognition, Poppler for PDF rasterisation, libmagic for content sniffing at ingest, and the OpenCV runtime libraries for the forensic metrics. The console image bakes only public configuration at build time, so the same image can be promoted between environments. The API's entrypoint waits for the database, applies migrations, and seeds only when explicitly asked — a production database is never populated with demonstration claims by a container restart.

6 Implementation

The implemented system is 18,725 lines of Python across 110 modules, 11,077 lines of TypeScript across 81 modules, and 2,039 lines of tests. This section documents each subsystem, concentrating on the decisions that were not obvious and on the measurements that had to be abandoned or reinterpreted. Detector parameters are collected in Appendix A.

6.1 Ingest and validation

Ingest is the one path in the system that must never be permissive and never be destructive. It proceeds in seven ordered steps, and the order is itself a control:

4. Stream the upload with a hard byte ceiling, aborting above the configured maximum rather than buffering an arbitrarily large body.
5. Sniff the content type from magic bytes. The declared MIME type and the filename are *evidence about the claimant's software*, not trusted input; anything outside PDF, JPEG, PNG and TIFF is rejected.
6. Validate the extension and record any disagreement with the sniffed type as a warning rather than a rejection — a legitimate document with a wrong extension should not be lost, but the discrepancy belongs in the record.
7. Parse structurally. A PDF that will not open, or an image that will not decode, is rejected as corrupt rather than stored, because an unparseable file cannot be analysed and storing it would create the appearance of coverage.
8. Check page count and decoded resolution against limits.
9. Compute the SHA-256 of the bytes and look for an existing document with the same hash. A duplicate is reported as a warning with the reference of the other copy; both copies are kept.
10. Write once to the `originals` scope — fsync, atomic rename, mode `0o440` — then create the document row. Filenames are sanitised and never used as storage keys.

The hash taken at this point is the document's identity for the rest of its life. Stage 1 of the pipeline re-reads the stored bytes and recomputes it; a divergence fails the stage. It is printed in the evidence report and returned in a response header when the original is downloaded, so a third party can confirm that the bytes they hold are the bytes that were analysed.

6.2 Rendering, OCR and field extraction

Pages are rasterised at 150 dpi with a 2400-pixel maximum edge into the artifacts scope, together with thumbnails. The render is the coordinate space in which every region in the product is expressed, which is why it is produced before recognition rather than on demand.

Recognition

Three recognition strategies sit behind one provider interface. The default **hybrid** provider tries a PDF's embedded text layer first and keeps it when it yields at least twelve words, falling through to pixel recognition otherwise, and records which route it took and why. The **text-layer** provider reads characters and coordinates directly with `pdfplumber`, clusters them into lines by vertical

position, scales PDF points into render pixels, and reports a fixed confidence of 0.995 — because it is exact character data, not recognition, and pretending to be uncertain about it would be dishonest in the other direction. The **Tesseract** provider handles scans and photographs, returning per-word text, boxes and confidences. A **fixture** provider replays stored results for offline demonstrations.

Schema-driven extraction

Extraction is declarative. A library of 60 field specifications — each with a label synonym list, value patterns, a value type and a group — is composed into 12 document-type schemas that declare which fields are expected and which are *required*. Requiredness is a property of the type, not of the field: a patient name is required on a medical invoice and meaningless on a vehicle registration. The `other` type declares no required fields at all, because an unclassified document has no contract to violate.

The engine builds a view of each recognised line with character spans, per-span confidence and a column-gap threshold derived from the median glyph height, then matches labels. The non-obvious part is what counts as a label match, and it took three iterations to get right. A candidate label counts only when a separator character follows it, or the label owns the whole line — as in a stacked form or an identity card — or a column gap at least as wide as the threshold appears both before and after it. Without that third condition the phrase *MEDICAL INVOICE* in a letterhead is read as the label “Invoice”, and the line *Surgeon fee – laparoscopic procedure 1 1,850.00* is read as the label “procedure”. Both failures were observed and both are now regression tests.

Values are trimmed by type, cut short when the next known label appears on the same line, and may fall to the following line only for fields that declare it, at a confidence penalty. A confidence is composed from the recognition confidence, the quality of the label match, the next-line penalty and whether the value normalises to its declared type; organisation names may be recovered from the first line of page one only when it matches that field's own marker family. Line items are matched as a description followed by two or three numbers, skipping rows containing total, tax or discount terms. Every value is persisted with its raw text, its normalised form, its type, its confidence, its page, its bounding box and its provenance — one of recognition, next-line recognition, pattern, letterhead or inference.

> **Why every value carries a box**
>
> The bounding box is not a convenience for the interface; it is what makes a finding checkable. A consistency rule that reports “the printed total does not match the sum of its own line items” is only useful if the reviewer can see *which* printed total the rule read. Without coordinates, the rule's claim is unfalsifiable from the reviewer's position, and an extraction error becomes indistinguishable from a document defect.

6.3 Forensic detectors

Four detector families measure the file itself. Their thresholds, evidence keys and confidence functions are tabulated in Appendix A; this section records the three decisions that shaped them.

Statistical significance, not per-block thresholds

The first implementation of error-level analysis and noise-residual analysis computed a per-block z-score and flagged outliers. On claim documents this does not work in either direction. A document page is mostly blank paper, and its content blocks vary enormously — body text, a logo, a signature, a photograph — so no single block is ever a significant outlier against the page as a whole, while lowering the threshold floods the report with the pre-printed furniture of the template.

The implemented approach establishes region significance statistically. A content mask first excludes blank paper by local standard deviation. A sliding window then runs a two-sample comparison of the window against the remainder of the page's content blocks, and a region is reported only when it clears both a z threshold and a minimum effect size, with overlapping same-direction hits merged. Reporting is capped at the strongest two or three regions per metric, because a list of twenty regions is not an investigative lead.

Two detectors measured but deliberately not raised

Resampling periodicity and JPEG lattice energy are standard image-forensic measurements and both are implemented. On text-heavy rasters both are confounded by the subject matter: printed text has a regular line pitch and crisp stroke gradients, which produce precisely the periodicity and the block-edge energy these metrics look for. Measured on a document image, they fire on essentially every input.

Both are therefore computed and retained in the evidence blob as context, flagged `signal_suppressed_for_text` with the reason, and suppressed as signals once text dominance passes a threshold. A detector that fires on every input is worse than no detector, because it trains the reviewer to ignore the category — but deleting the measurement would also delete the reviewer's ability to see that it was considered.

Pixel forensics do not run on vector page renders

Error-level analysis, noise residual, compression history and resampling carry information only where there is a real capture or compression history. A page rendered from vector PDF content has none: measuring it measures the renderer. For PDFs the raster suite therefore runs over *embedded* raster objects — where a scan or a pasted graphic actually lives — and when a page has no embedded rasters the stage records `not_applicable` explicitly rather than reporting noise. Structural, metadata, font and typography analysis carry the forensic signal for digital PDFs, and it is there that the strongest single measurement in the platform sits.

Exact per-character font analysis

For digital PDFs the typography detector does not estimate. It reads the font resource each character was drawn from, clusters characters into lines by baseline with a size-proportional tolerance, and reports a line in which a minority of characters come from a different font resource than the dominant one. This is an exact structural fact about the file rather than a statistical inference about pixels, which is why it carries the highest source reliability in the correlation layer

and one of the highest weights in the risk model. It is the measurement that identifies a re-typed total: *“Font change inside a text line: “$8,500.00” (page 1)”*.

6.4 Tampering correlation

Tampering is implemented as a correlation layer rather than a detector, because any single forensic metric on a document page is noisy. Overlapping suspect regions emitted by the independent detectors are merged with a small slack, and their confidences are combined by a noisy-OR damped by each source's reliability:

```
combined = clamp( 1 − Π ( 1 − confidence_i × reliability_i ), 0, 0.93 )
```

Source reliabilities run from 0.90 for an exact PDF font mix down to 0.48 for an isolated low-confidence recognition island (Appendix A.6). A region is reported only when a high-reliability source fires **or** at least two independent sources overlap on the same area, and the combined confidence clears 0.45.

The region is then *named* by intersecting it with the extracted fields, which is what turns a measurement into a sentence an investigator can act on. The tamper type follows from the field's value type where the region covers a field — money gives an edited amount, a date gives a modified date, an identifier gives an altered identifier, a party name gives a replaced name — and from the contributing source where it does not. Thirteen tamper types are produced in total. The region comes from measurement; the label comes from extraction; the finding records both, along with which detectors agreed, their individual confidences and boxes, and the reliability table used.

6.5 The consistency rule engine

Twenty deterministic rules over the extracted fields are the most trustworthy signals in the platform — subject to extraction being correct, which is why every rule finding carries the fields it used, their values and their own confidences and boxes. Rules are declarative: each states its code, name, description, default severity, weight, the document types it applies to, and its parameters, all of which an administrator can override at run time.

Table 13. The twenty consistency rules by family. Severities and weights are the seeded defaults.

Family	Rules
Arithmetic	`LINE_ITEM_SUM_MISMATCH` (high), `TOTAL_ARITHMETIC_MISMATCH` (high), `TAX_CALCULATION_MISMATCH` (medium), `BALANCE_MISMATCH` (medium), `AMOUNT_PAID_EXCEEDS_TOTAL` (medium), `NON_POSITIVE_TOTAL` (medium), `DUPLICATE_LINE_ITEMS` (low)
Dates	`TREATMENT_AFTER_INVOICE` (high), `ADMISSION_AFTER_DISCHARGE` (high), `SERVICE_OUTSIDE_POLICY_WINDOW` (high), `DOCUMENT_EXPIRED_AT_EVENT` (medium), `FUTURE_DATED_DOCUMENT` (medium), `IMPLAUSIBLE_DATE_OF_BIRTH` (medium)

Family	Rules
Identity and format	`VIN_CHECK_DIGIT_INVALID` (high, ISO 3779), `IBAN_CHECKSUM_INVALID` (high, ISO 13616 mod-97), `MISSING_REQUIRED_FIELD` (medium), `CURRENCY_AMBIGUOUS` (medium), `IDENTIFIER_FORMAT_UNEXPECTED` (low), `INTERNAL_NAME_MISMATCH` (low), `LOW_CONFIDENCE_CRITICAL_FIELD` (low)

Two implementation properties matter more than the rule list. First, **every rule reports three outcomes**: failed, passed, or not applicable with the reason. The console displays all three. A green tick is what makes a Low Concern assessment meaningful rather than merely empty — "twelve checks were applicable and eleven passed" is a finding; "no findings" is an absence that could equally mean nothing was checked. Second, **a rule that raises is isolated**: the outcome is recorded as an error for that rule alone and logged, so a defect in one rule cannot fail the stage or the run. That behaviour has its own test, using a deliberately broken rule.

6.6 Cross-document comparison

Seventeen fields are compared across the documents of a claim and against the claim record, with a comparison mode per field: fuzzy token-set matching for names, exact-after-normalisation for identifiers, day granularity for dates, and a proportional tolerance for money.

The substantive problem here is not comparison but *deciding what may be compared*. Comparing every field across every pair of documents manufactures findings from legitimate differences: two separate invoices in one claim are not expected to carry the same total, and a dependant's date of birth is not expected to match the policyholder's. Each comparable field therefore declares a **pairing policy**.

Table 14. Pairing policies. A comparison outside its policy is recorded as skipped, with the reason.

Policy	Fields	Rule
`any`	policy number, claim number, patient / insured / customer name, provider name, incident date, vehicle registration, VIN, currency	Compare across every pair of documents holding the field.
`same_party`	date of birth, national identifier, diagnosis, treatment, admission and discharge dates	Compare only when both documents record the value for the same individual, resolved by subject.
`same_invoice`	total amount	Compare only when both documents cite the same invoice, or one is a settlement document for the other.

Subject resolution is what makes the `same_party` policy work: a date of birth on a hospital invoice belongs to the *patient*, not to the policyholder who submitted the claim. Same-person matching is deliberately strict and order-sensitive, because a shared surname is not identity — "Daniel Okonkwo" against "Amelia R. Okonkwo" scores 0.73 against a 0.80 threshold and is correctly

treated as a different person. Skipped comparisons are still recorded and displayed with a human-readable reason, so the reviewer can see what was *not* compared and why; that list is as much a part of the evidence as the mismatches.

6.7 Synthetic-media analysis

The bundled provider is a deterministic, fully explainable eight-feature detector — **not a trained classifier**, and labelled as such everywhere its output appears. It measures spectral high-frequency share, noise-field uniformity, cross-channel detail correlation, edge-gradient kurtosis, capture-provenance completeness, text-rendering irregularity, block-variance entropy and colour-palette density, with published weights summing to one (Appendix A.7). Scores for a lossless page render are damped, since a vector-rendered page is not photographic.

Output is always a band: no significant indicators, weak, possible, or notable. There is no code path that can emit “definitely AI-generated” or a fraud verdict. The finding carries every raw measurement, the weight table, the damping factor and the provider's disclaimer. Consistently with its epistemic status, the synthetic-media category has the lowest ceiling in the risk model and a modest weight — a heuristic indicator should not on its own move a document into Review Recommended. The production seam is implemented and tested: it posts the page to a configured endpoint, expects a numeric likelihood with indicators, and raises rather than fabricating a score when the response is missing or malformed.

6.8 Trusted-source verification

Six verification routes sit behind one provider interface: policy, provider accreditation, invoice, identity, vehicle and settlement. The document type determines the request plan — a medical invoice triggers policy, provider and invoice lookups; a repair estimate triggers vehicle and policy — and each request is built from the extracted fields, backfilled from the claim record where the document is silent. A missing primary reference skips the request with a stated reason rather than sending an empty query.

Each provider returns named checks, each with what the source record holds, what the document says, and whether they agree. **Outcome grading distinguishes contradiction from missing detail**, which is the distinction that matters to an investigator:

Table 15. Verification outcome grading.

Outcome	Meaning
`match`	Every evaluable check agrees.
`partial_match`	A peripheral check disagrees, or no check could be evaluated.
`mismatch`	A **critical** check disagrees — the record exists and does not support the document. Examples: the policy is not active, the service date falls outside cover, the amount disagrees with the issuer's record, the identity document is reported lost, the vehicle carries a write-off marker.
`not_found`	No such record. Informative, but not a contradiction.

Outcome	Meaning
`unavailable`	The provider could not be reached. **Nothing is inferred** and the stage is recorded as degraded.

The bundled registry describes a fictional insurer, hospitals and authorities, and deliberately contains contradictions — a lapsed policy, an unaccredited clinic, an invoice whose issuer total differs from the document, a passport reported lost, a vehicle with a write-off marker, a returned payment — because a verification layer that always answers yes demonstrates nothing. Every demonstration response is labelled as such at five layers, and the provider mode is stored on the request itself, so a report generated today cannot be mistaken for a real confirmation when it is read next year.

6.9 The risk engine

The engine is a pure function of the findings, the verification outcomes, the rule weights and the stage-coverage map. Its design is documented in Section 5.5; three implementation properties are worth recording here.

- **Human decisions are first-class inputs.** A dismissed finding is excluded and the count of exclusions is returned with the score; a confirmed finding has its confidence promoted to at least 0.95 and its source recorded as human-confirmed. Recomputation after review is an explicit endpoint, so a reviewer sees the score change as a consequence of their own decision rather than as a background effect.
- **Degraded coverage is disclosed in the explanation.** If the recognition stage was unavailable, the assessment says so, and states that findings from the dependent areas are absent by omission. This is the difference between a low score that means “nothing was found” and one that means “little could be looked at”.
- **The model is data.** Weights, severities, rule parameters, category ceilings and band thresholds are rows an administrator can edit, and every edit is audited. The engine publishes the weights version with each assessment, so an old report remains interpretable against the model that produced it.

6.10 Evidence reporting

The PDF evidence report is generated server-side in ten sections: document information including the ingest SHA-256, claim information, processing information with the full stage table and provider snapshot, extracted information with line items, findings by family each tagged as measured, rule, model or human, synthetic-media detail with its provider and disclaimer, trusted-source verification with expected against actual and the demonstration marker where applicable, the risk composition with every signal's weight, confidence and contribution plus the published formula, the human review actions with who, when, what and why, and an audit-trail extract with the chain state. The report's own SHA-256 and the document's SHA-256 are both recorded on the report row, and the report snapshots a chain verification at the moment of generation.

6.11 The investigator console

Twelve authenticated screens are gated by the same permission matrix the API enforces; an item a role cannot use is not rendered. The dashboard opens the working day with queue health, risk distribution, daily intake banded by risk, findings by detector family split into awaiting and adjudicated, verification outcomes, and the cases that need a person next.

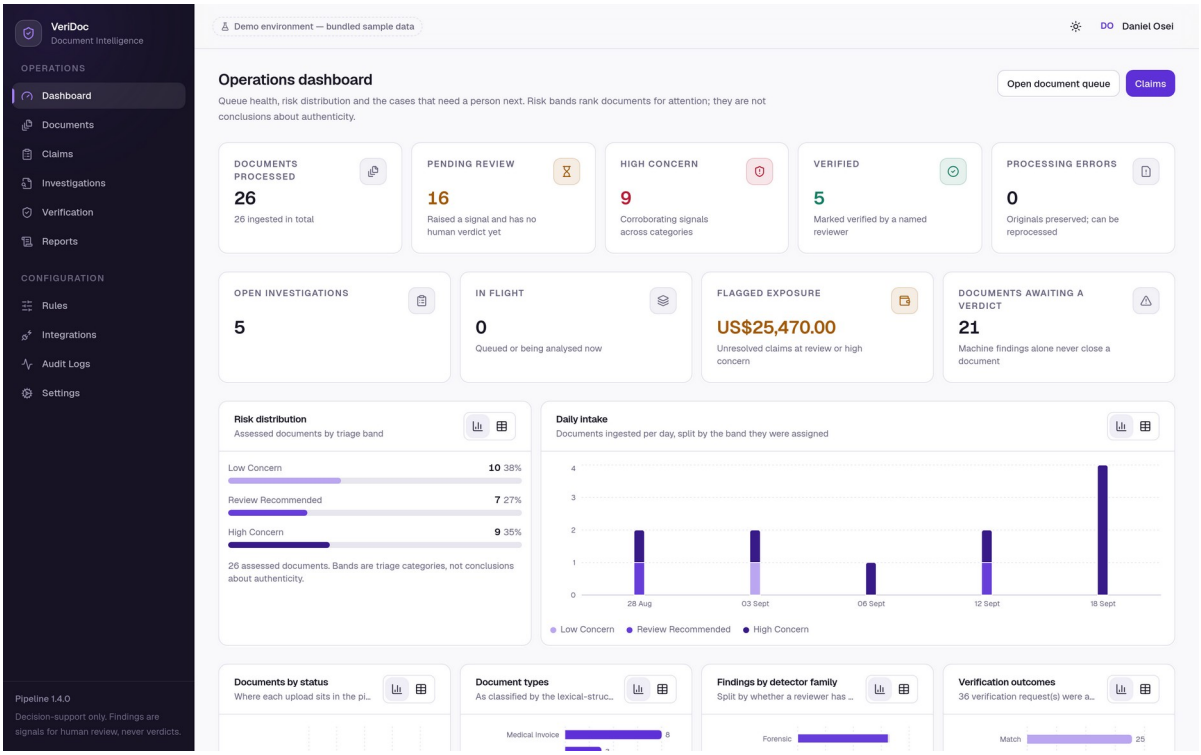


Figure 1. The operations dashboard. Counters are operational rather than decorative; every chart can be switched to the table of rows behind it.

The document queue is the triage surface: reference, filename, detected type with the classifier's confidence, claim, risk band with score, finding counts, pipeline state and human verdict, filterable by band, status, type and verdict and searchable by reference, filename or SHA-256.

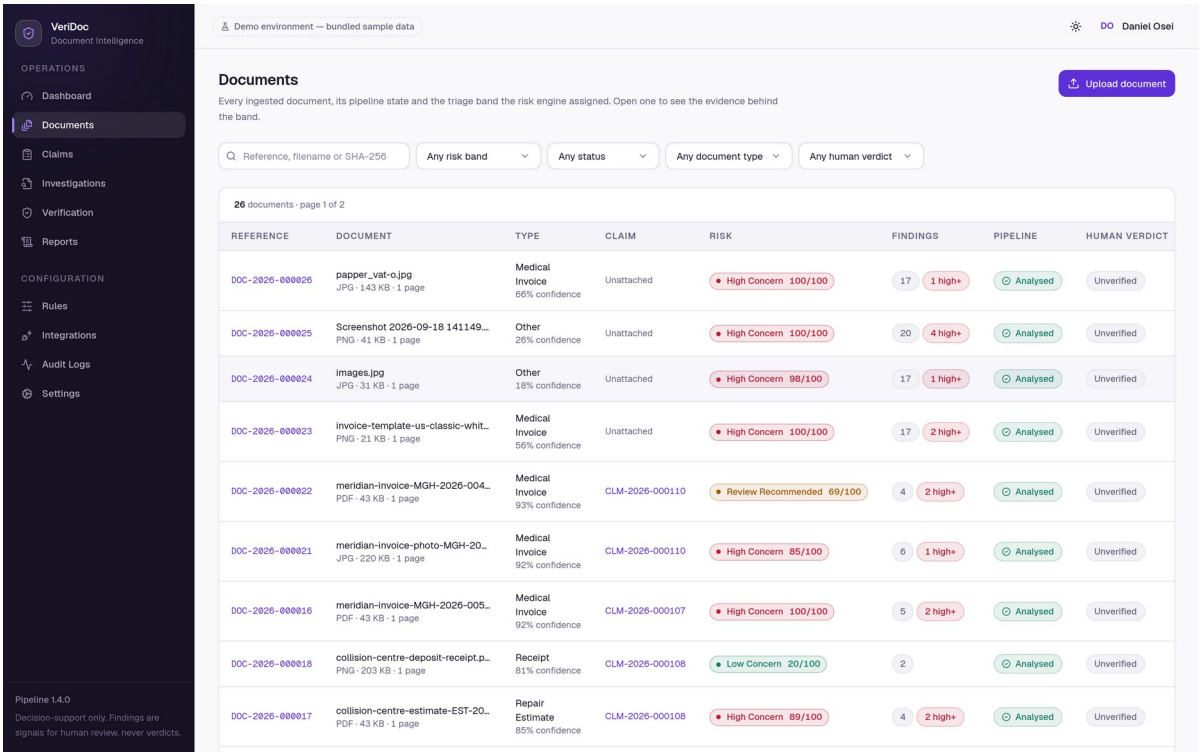


Figure 2. The document queue. The risk band is always shown with its name and its score, never as a colour alone.

The workstation is where the product's central claim is made good: every panel on the right can box the pixels it is discussing on the left. The overview states the band, the score against the band thresholds, a plain-language explanation naming the principal contributors, and the interpretation note; the “why this score” list gives each signal's contribution with a control that highlights it on the page.

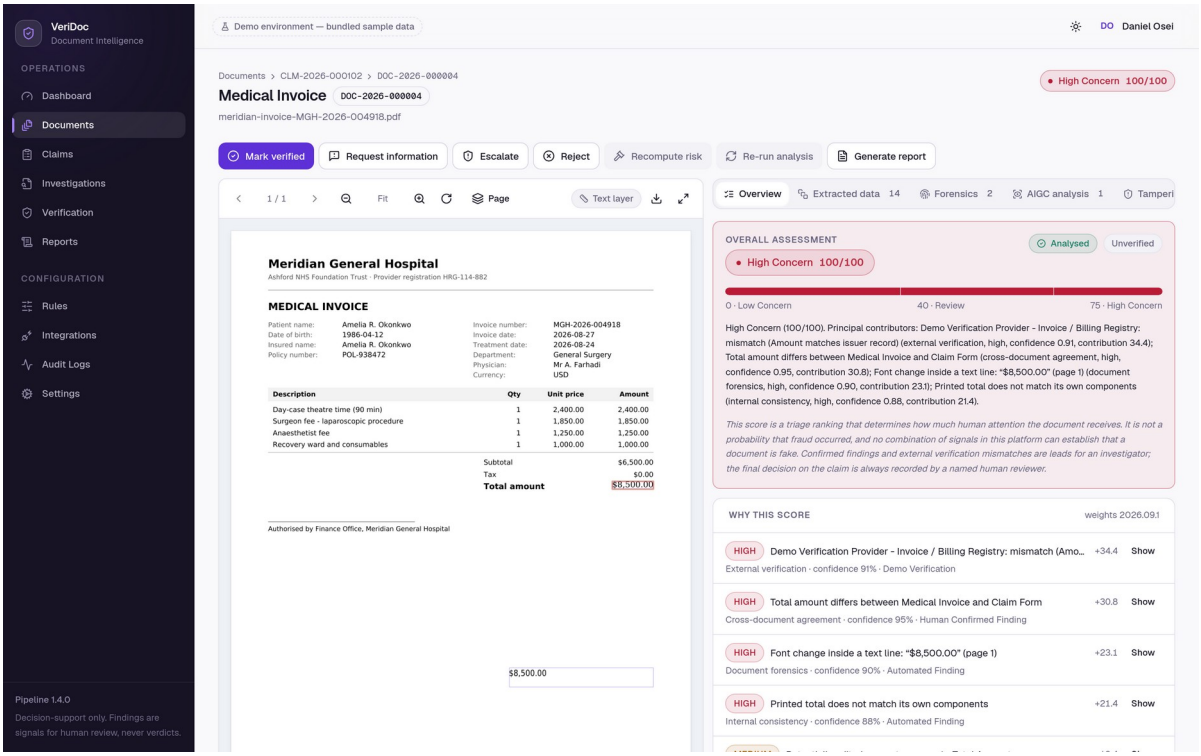


Figure 3. The investigation workstation on a document with an edited total. The printed total of $8,500.00 is boxed on the page; the subtotal of its own line items is $6,500.00; the issuer's record and the claim form both state 6,500.00.

Eight further panels carry the recognition detail and extracted fields, the forensic findings with the full metadata table and the raw measurements including metrics that did not fire, the synthetic-media indicators with their contributions, the tampering correlation with the detectors that agreed, the consistency rules with passes and non-applicable outcomes, the cross-document matrix including the comparisons deliberately not made, the verification checks with their disclaimers, and the document's audit trail.

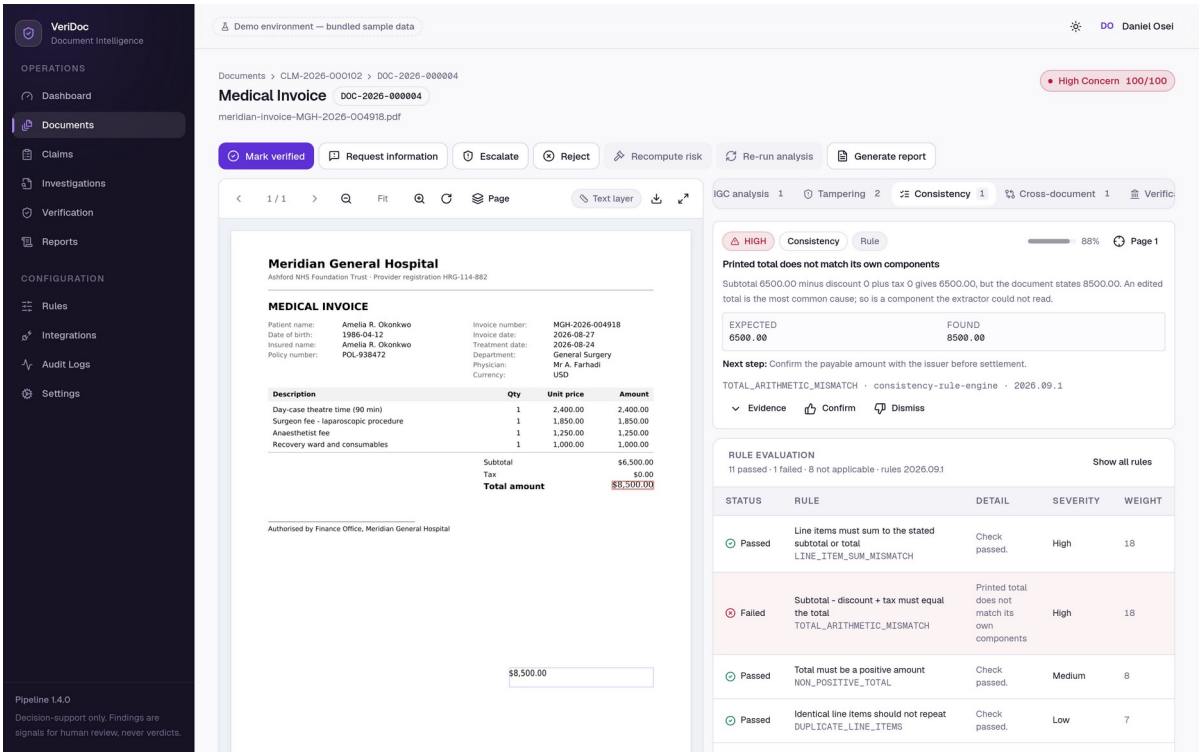


Figure 4. The consistency panel. Passed and not-applicable checks are displayed alongside failures, so a clean result is evidence rather than an absence.

Adjudication is deliberately effortful. Confirming or dismissing a finding opens a dialog that states the consequence and requires a written reason of at least ten characters, enforced by the server as well as the interface; the reason is recorded against the reviewer's name in the audit trail and survives reprocessing. A claim decision requires a longer reason, and the server refuses an approval while any high or critical finding remains unadjudicated — machine findings must not become a decision by default.

The administration screens make the model inspectable: the rules screen shows the category ceilings, the band thresholds and every rule with its severity and weight, editable in place by an administrator and read-only for everyone else; the integrations screen lists each provider's mode and the *names* of the environment variables it needs, never their values; the audit screen verifies the hash chain at the top of the page and names the first divergence if there is one.

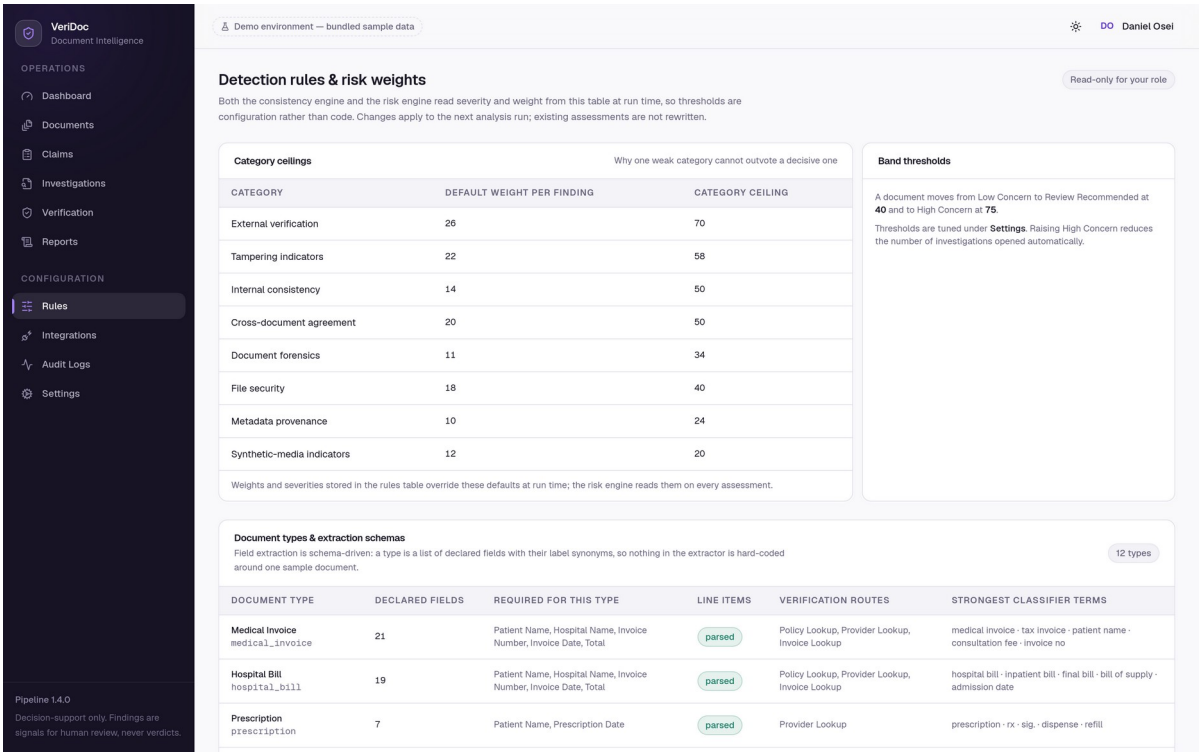


Figure 5. The detection-rules screen. Thresholds are configuration held in the database, not constants in code, and every change is audited.

6.12 Interface revision

A late revision addressed three defects in the console's presentation layer that had accumulated during feature work. They are recorded here because two of them were accessibility defects rather than matters of taste.

- **Typeface.** The Tailwind configuration referenced font-family CSS variables that were never defined, so the entire console had been rendering in whichever interface font the operating system supplied. A self-hosted variable typeface (Geist Sans and Geist Mono, shipped as local font files rather than fetched from a content delivery network) is now declared and loaded, which also removes a third-party request from a page that displays medical documents.
- **Type scale.** Body text throughout the console was 11 px, below a reasonable floor for all-day reading. The two smallest steps of the scale were re-stepped to 12 px and 13 px, which lifted every existing usage at once rather than requiring sixty files to be edited, and a proper heading ramp with optical letter-spacing was added.
- **Palette.** The navy-blue enterprise palette was replaced by a single tinted neutral ramp with an iris accent, and the three risk colours were re-tuned. The re-tuning was validated computationally rather than by eye (Section 7.4): all three band colours now exceed the 4.5:1 contrast requirement for text on the card surface, and the triad's worst pair separation under simulated deuteranopia improved from ΔE 2.9 to ΔE 9.5 against a floor of 8.

The sign-in screen was also reworked, and the demonstration-account panel changed to list role names and email addresses only. Previously the shared demonstration password was printed on the page and pre-filled into the password field; it is now neither shown nor pre-filled, and selecting a role fills the email field alone. The change is small but it removes a habit — a credential displayed

on a sign-in page — that has no place in a system holding medical records, even in a demonstration deployment.

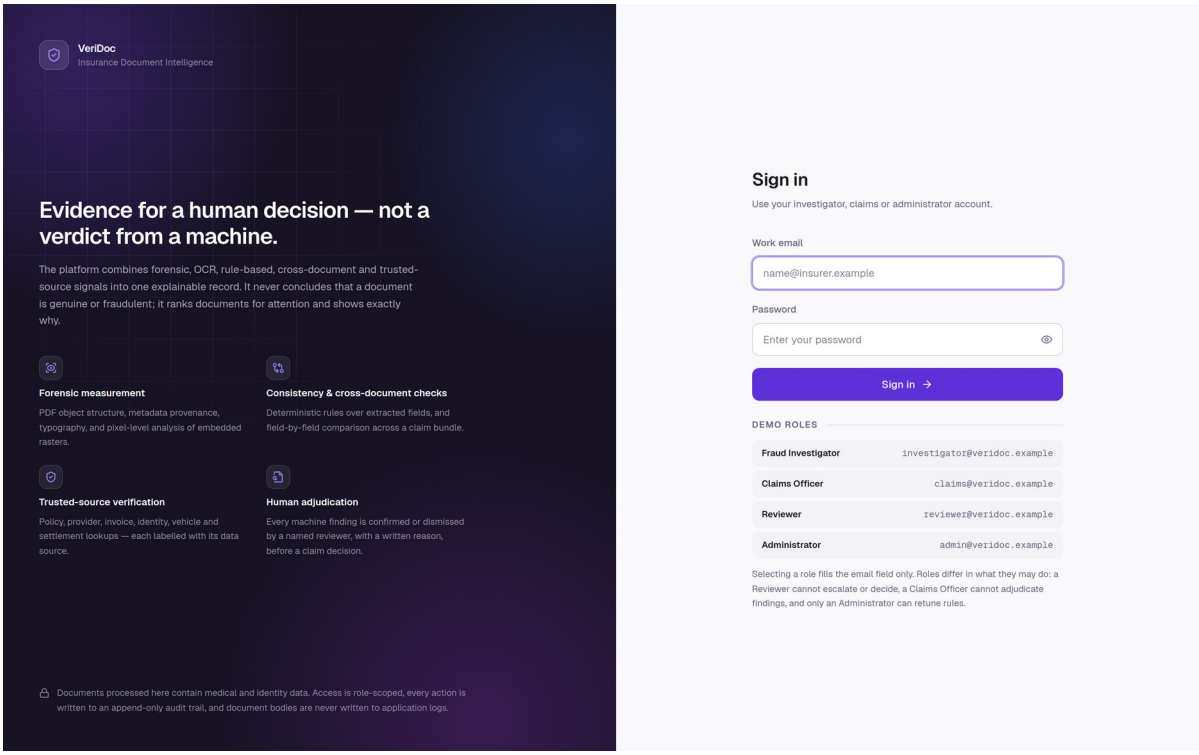


Figure 6. The sign-in screen as deployed. No password is displayed or pre-filled; the demonstration panel lists role names and addresses only.

7 Testing and Validation

7.1 Test strategy

The suite runs against a **real PostgreSQL database and real object storage**, not against mocks. That is a deliberate and slightly expensive choice, justified by what is actually worth testing in this system: schema constraints, the audit hash chain, storage permissions and scope isolation, and the analysis pipeline end to end. None of those is meaningfully exercised against a substitute. A test database is created automatically, its schema is rebuilt for the session, and reference data is seeded; only external providers are substituted, and the demonstration documents used as fixtures are generated by the same code that produces the sample corpus, with fixed seeds.

Four principles shaped the test content. Tests assert **wording** as well as behaviour, because the platform's central commitments are linguistic: there are tests that the risk explanation never asserts a probability of fraud and that synthetic-media output is always a band. Tests assert **negative** behaviour: that unrelated invoices are *not* compared on total, that a dependant's date of birth is *not* compared against the policyholder's, that a rule returns not-applicable rather than passing when its inputs are absent. Tests assert **degradation**: that a provider outage marks one stage and preserves the document. And tests assert **authorisation per role and per endpoint**, because a permission matrix that is not tested is a permission matrix that is wrong.

7.2 Automated test suite

The suite contains 155 test functions across 10 files, which expand with parametrisation to **174 executed tests**. The complete run as submitted:

```
        $ cd backend && pytest
        ........................................................................ [
41%]
        ........................................................................ [
82%]
        ..............................
[100%]
        174 passed in 157.78s (0:02:37)
```

Table 16. Test suite composition and coverage areas. Appendix D lists the assertions per file.

File	Tests	Area covered
`test_rules_engine.py`	20	Every consistency rule in isolation, pass and not-applicable reporting, severity and weight overrides, document-type scoping, and isolation of a deliberately broken rule.

File	Tests	Area covered
test_risk_and_aigc.py	19	Monotonicity, severity effect, category saturation, dismissal and confirmation handling, configured thresholds, mitigation suppression over severe findings, the wording guarantees, and synthetic-media determinism and labelling.
test_api_workflow.py	18	The whole investigator workflow over HTTP: upload, filtering, evidence panels, signed links and their rejection when tampered, reason enforcement, reassessment, role limits, approval blocking, verification, notes, report generation and the audit trail.
test_crossdoc_verification.py	17	Pairing policies, dependant handling, claim-record reconciliation, all six demo providers, outcome grading, and the unconfigured production provider.
test_storage_and_errors.py	17	Path-traversal rejection, scope isolation, read-only originals, artifacts purgeable without touching evidence, signed-resource binding and expiry, token type enforcement, error contracts and security headers.
test_audit_and_admin.py	16	Chain linkage, detection of rewritten history, redaction before storage, rule and provider and threshold administration, and that providers never expose credential values.
test_auth_rbac.py	14	Sign-in, non-disclosing failure wording, token types and refresh rotation, the full permission matrix and per-endpoint authorisation.
test_upload_security.py	14	Executables, extension spoofing, corrupt and oversized uploads, EICAR quarantine with the file preserved, hash fidelity and duplicate detection.
test_extraction.py	11	Recognition routing, money, date and name normalisation, same-person strictness, classifier behaviour, extraction with boxes, and the label-in-heading regression.
test_pipeline_e2e.py	9	Clean and tampered documents end to end, provider outages, review carry-forward across reprocessing, artifact storage, and vector-PDF stage skipping.

7.3 Requirement walkthrough

Unit and integration tests establish that components behave; they do not establish that the *product* does what it claims. A scripted walkthrough therefore exercises all twenty product requirements against a live instance, asserting the outcome of each step. It signs in as three roles, creates a claim, generates and uploads a document with planted anomalies, waits for the queue, reads every evidence panel through the API, runs verification, adjudicates findings, escalates, adds notes, generates the report and verifies the audit chain. The run as submitted passes 23 of 23 checks, including the two negative controls that matter most: that confirming a finding without a reason is refused, and that approving a claim with unadjudicated high findings is refused.

7.4 Accessibility validation

Design principle P6 — colour never carries meaning alone — was validated computationally rather than by inspection, because colour-vision separation is not reliably assessable by eye.

The three risk-band colours are used as badge text, so the binding requirement is the 4.5:1 contrast ratio for text. Measured against the card surface: Low Concern 5.00:1, Review Recommended 5.29:1, High Concern 6.35:1, with the dark-theme steps chosen separately rather than derived by inversion and measured at 8.45:1, 9.81:1 and 5.63:1. Separation under simulated colour-vision deficiency was computed as a perceptual distance in OKLab: the triad's worst pair improved from ΔE 2.9 to ΔE 9.5 under deuteranopia, clearing the ΔE 8 target.

Charts do not use the band triad at all. Risk bands are ordered, so in charts they are encoded as a **single-hue ordinal ramp** validated for monotone lightness, an adjacent lightness step of at least 0.06, a single hue and a light end that clears the chart surface at 2.12:1 in the light theme and 2.35:1 in the dark. The semantic triad is retained on badges and chips, where the band's name is always present. In addition, every chart in the console can be switched to the table of rows behind it, which is the relief for colour-carried information and is also what an investigator wants when a number has to be quoted in a case note.

7.5 Static analysis and build verification

- `ruff` lints the backend; `alembic check` fails if the models have drifted from the migrations, so the schema cannot silently diverge from the code.
- `tsc --noEmit` type-checks the console under `strict` mode — clean as submitted.
- `next build` produces the production bundle — clean as submitted, with both self-hosted typefaces confirmed present in the build output.
- Both container images build from a clean checkout, and the deployed stack reports healthy on the database, Redis, the API and the console.

8 Results and Evaluation

8.1 The evaluation corpus

The seeded corpus is 10 claims and 22 generated documents covering nine scenario classes, with scripted human-review history. Every anomaly is present in the file at the object or pixel level (Section 4.4), so the findings below are measurements rather than fixtures.

Table 17. Evaluation scenarios and the evidence the platform recovers from each.

Scenario	Claim	What the platform finds
Clean legitimate document	CLM-2026-000101, -000106	Everything reconciles; applicable checks pass and are displayed as passes; Low Concern.
Edited invoice total	CLM-2026-000102	The total is drawn from a different font resource than the rest of its line, sits off its baseline and is overlaid by an annotation; it does not match the sum of its own line items.
Metadata anomaly	CLM-2026-000105	Producer names a raster editor; modification precedes creation; an incremental revision is present; bytes follow the final end-of-file marker.
Cross-document mismatch	CLM-2026-000102	The invoice states 8,500.00 where the claim form and the claim record both state 6,500.00.
Potential synthetic image	CLM-2026-000108	A smooth colour field with no sensor noise and no capture metadata yields “potential synthetic-media indicators” — a band, with its eight contributions shown.
Verification mismatch	CLM-2026-000104, -000109	A lapsed policy outside its cover window, an unaccredited clinic, a returned payment, a vehicle write-off marker and a passport reported lost.
Tampered photograph	CLM-2026-000103	The amounts block was re-typed after capture: error-level analysis, copy-move and a named tampering region over the affected field.
Duplicate re-upload	CLM-2026-000110	A byte-identical file already on record under another claim, attributed to the later submission.
Invalid VIN and line-item sum	CLM-2026-000108	The ISO 3779 check digit fails and the line items do not sum to the printed total.

8.2 Worked example

Document `DOC-2026-000004` in claim `CLM-2026-000102` is a one-page medical invoice from a fictional hospital whose printed total was re-typed from 6,500.00 to 8,500.00. It is worth tracing

because five independent subsystems reach the same conclusion by different routes, and because the score reaches its ceiling without any single detector having asserted anything about fraud.

Table 18. Principal risk contributions for DOC-2026-000004, as returned by the risk engine.

Signal	Category	Conf.	Contribution
Invoice / billing registry: amount does not match the issuer's record	External verification	0.91	34.4
Total amount differs between the medical invoice and the claim form	Cross-document agreement	0.95	30.8
Font change inside a text line: “$8,500.00” (page 1)	Document forensics	0.90	23.1
Printed total does not match the sum of its own components	Internal consistency	0.88	21.4

The four routes are epistemically distinct, and that is the point. The verification mismatch is an external record contradicting the document. The cross-document mismatch is two documents in one bundle disagreeing. The font change is an exact structural fact about the file. The arithmetic failure is a deterministic property of the document's own printed values. A tampering finding then names the region — the region from measurement, the label from extraction — and the viewer boxes the pixels, which is what the investigator actually inspects.

The assessment is High Concern at 100/100, and it is accompanied, as every assessment is, by the note that the score is a triage ranking and not a probability that fraud occurred. What the platform has established is that the printed total is not supported by the document's own components, by the other document in the bundle, or by the issuer's record, and that the characters of that total were drawn from a different font resource than their line. What it has not established — and says so — is who did that, when, or with what intent.

Dismissing the cross-document finding with a written reason and recomputing demonstrates the review path end to end: the signal is excluded, the exclusion is reported with the new score, the finding is retained with its reason, and the decision survives a subsequent re-run of the analysis.

8.3 Behaviour on clean documents

The corpus deliberately includes documents with nothing wrong with them, because a detection platform is judged as much by its behaviour on clean inputs as by its behaviour on manipulated ones. On these the platform reports Low Concern with the applicable checks listed as *passed*, the non-applicable ones listed with the reason they did not apply, and the informational findings — a stripped EXIF block on an exported image, a low-quality JPEG — shown in a separate “checks that produced no concern” section rather than as signals.

Two behaviours are worth noting as results in their own right. A heavily compressed image raises an informational finding stating that the compression has destroyed the detail the other detectors

need, so an absence of findings on that document is uninformative — the platform reports the weakness of its own coverage. And a vector PDF records the pixel-history stage as not applicable rather than producing measurements of the renderer.

8.4 Evaluation against the objectives

Table 19. Objectives against outcomes.

#	Outcome	Evidence
O1	Met	Seven-step validated ingest; write-once originals at mode 0o440; hash re-verified in stage 1; quarantine without deletion; 14 upload-security tests.
O2	Met	Three recognition strategies behind one interface; 12 schemas over 60 field specifications; a box and a confidence on every value; 11 extraction tests.
O3	Met	Four detector families over PDF structure, metadata, pixels and typography, including exact per-character font resources; two confounded metrics measured and explicitly not raised.
O4	Met	Noisy-OR correlation over eleven ranked sources with an agreement gate; 13 tamper types named by intersecting the region with extracted fields.
O5	Met	20 declarative rules with administrator overrides; passes and not-applicable outcomes reported; per-rule isolation; 20 tests.
O6	Met	17 fields compared with three pairing policies and subject resolution; skipped comparisons displayed with reasons; 17 tests.
O7	Met, within the stated scope	Eight-feature deterministic provider with published weights, band-only output, demo labelling at five layers, and a tested production seam. A trained classifier remains out of scope (Section 2.3).
O8	Met, within the stated scope	Six routes behind one interface with critical-versus-peripheral outcome grading. Live registry connections require commercial access (Section 2.3).
O9	Met	Published three-step formula, per-category saturation and ceilings, database-held weights and thresholds, full term-by-term output; 19 tests including monotonicity and saturation.

#	Outcome	Evidence
O10	Met	Split-screen workstation with bidirectional finding-to-pixel linkage; mandatory written reasons enforced server-side; explicit reassessment; decisions preserved across reprocessing.
O11	Met	Hash-chained audit trail with a verification endpoint; four roles over 33 permissions enforced server-side; deployed behind TLS with the API unreachable from the internet.

What these results do not establish

No detection rate, false-positive rate, precision or recall is claimed anywhere in this report. The evaluation corpus was generated by the same project that wrote the detectors, which makes it valid for demonstrating that each detector fires on the property it claims to measure and that the system works end to end, and invalid for estimating performance on real claim traffic. Obtaining such estimates requires a labelled corpus of genuine claim documents with adjudicated outcomes, and is the first item of future work.

8.5 Operational characteristics

Table 20. Measured operational characteristics of the submitted system.

Property	Value
Full test suite runtime	157.78 s for 174 tests, including the end-to-end pipeline tests that render and recognise real documents
Analysis coverage limits	30 pages recognised, 12 pages pixel-analysed, 60 pages and 25 MB accepted per document
Render resolution	150 dpi, maximum edge 2400 px
Queue behaviour	RQ worker when Redis is reachable; otherwise an in-process background task, with the path taken recorded on the run
Access token lifetime	45 minutes, with a single transparent refresh through the console proxy; refresh token 7 days
Signed document links	5-minute default lifetime, bound to resource, purpose and user
Deployment footprint	Five containers on one host; no host ports published; only the console routed
Deployed state	Live at https://veridoc.rmaikr.top with database, cache, API and console all reporting healthy

9 Limitations, Ethical and Legal Considerations

This section is longer than is customary, deliberately. A platform that examines documents used to decide whether a person's medical bill is paid has obligations that are not discharged by working correctly.

9.1 What the platform cannot establish

The platform cannot establish that a document is genuine, and it cannot establish that a document is fraudulent. No combination of the signals it measures supports either conclusion, for a reason that is structural rather than a matter of current maturity: every measurement it takes has innocent explanations that occur commonly in legitimate claim traffic. Nor can it establish intent, authorship or the time of an alteration. The risk score is a ranking of how much human attention a document should receive, and the interpretation note saying so is emitted by the engine with every assessment.

Three narrower limits follow. An **absence of findings is not evidence of integrity** — it may mean the document was heavily compressed, or that a stage was degraded, both of which the platform discloses. **Extraction is a dependency of the deterministic rules**: a misread total produces a real arithmetic failure on a sound document, which is why every rule finding carries the fields it used with their own confidences and boxes. And **the platform sees one claim at a time**: organised fraud whose signal lies in patterns across claims is outside its view entirely.

9.2 Detector-level limitations

Each detector's limitations and benign explanations are documented in the repository's detector catalogue and surfaced in the product. The cases most likely to mislead a reviewer are collected here.

Table 21. Signals most frequently produced by benign causes.

Signal	Common innocent explanation
Producer names a raster or design editor	A clinic cropped, rotated or downscaled a scan before sending it.
Incremental PDF revision present	Digital signatures and form filling append revisions by design.
Annotation drawn over the page	Reviewer comments, approval stamps and legitimate redaction are placed exactly this way.
EXIF stripped	Screenshots, exports and every major messaging application remove it.
Error-level analysis region anomaly	A logo, an embedded photograph, a pre-printed block or a stamp.
Copy-move region	Repeated template graphics, duplicated stamps, tiled backgrounds.
Mixed image compression	A lossless logo beside a JPEG photograph.
Masked image overlay	Signatures and stamps are composited precisely this way.
Glyph size or stroke weight anomaly	Deliberate emphasis, bold text, ink bleed.
Baseline anomaly	Uneven scanning, page skew, subscripts.

Signal	Common innocent explanation
Low-confidence recognition island	A smudge, a fold, or a stamp overlapping text.

9.3 Demonstration providers

The single most damaging thing this platform could do is present demonstration output as a real external confirmation. Six of the trusted-source providers and the synthetic-media provider answer from bundled data. The countermeasure is defence in depth rather than a disclaimer in a manual: the demonstration flag is stored on the verification request and on the result, the disclaimer text naming the bundled registry is stored with the answer, the console shows a badge beside every such result, the pipeline stage message names the demonstration providers used, the resulting risk signal records its source as demonstration verification, and the PDF report marks the section `[EXTERNAL — DEMO]`. A report generated today therefore remains correctly labelled when it is read next year by someone who knows nothing about how the system was configured.

9.4 Personal and special-category data

Claim documents contain health data, identity-document numbers, dates of birth, addresses and bank details — in European terms, special-category data under Article 9 of the General Data Protection Regulation. The design consequences implemented here are data minimisation in what is extracted and retained, redaction before audit rows are written, separation of originals from derived artifacts so that retention can purge the latter without touching evidence, an audited record of every document view so that a subject-access request can be answered precisely, and the absence of document bodies from every log.

The audit-redaction decision is worth stating explicitly because it trades one good against another. Audit entries store structured field differences with secrets and special-category fields dropped before insertion. That loses some forensic detail about *what changed*. It is the right trade: the claim record already holds the customer's details under normal access control and normal retention, and duplicating them into an append-only store retained for years would widen the exposure of the most sensitive fields in the system for no accountability benefit. The audit trail records who did what, not the contents of a medical document.

Two obligations are noted as **not discharged by this project** and belonging to any real deployment: a lawful basis and a data-protection impact assessment for the processing, and encryption at rest. The storage interface exposes no plaintext caching layer, so enabling server-side encryption on the object store requires configuration rather than code, and database encryption is a platform concern — but neither is configured in the demonstration deployment.

9.5 Automation bias and the human in the loop

The most serious ethical risk in a system of this kind is not that it is wrong. It is that it is right often enough for a reviewer under time pressure to stop checking — at which point “decision support”

has become automated decision-making with a human signature on it. Article 22 of the GDPR is relevant precisely because a rubber-stamped output is a solely automated decision in substance.

Five mechanisms in the implementation are aimed squarely at this risk:

11. **Adjudication is effortful by construction.** Confirming or dismissing a finding requires a written reason of at least ten characters, enforced server-side, recorded against the reviewer's name, and displayed thereafter beside the finding. A reviewer who has typed a reason has engaged with the finding.
12. **Approval is refused, not warned against,** while any high or critical finding remains unadjudicated. The refusal is in the API, not the interface, so it cannot be clicked past.
13. **Separation of duties** prevents the same person from establishing a finding and disposing of the claim: an investigator adjudicates findings but cannot decide a claim; a claims officer decides but cannot adjudicate.
14. **Uncertainty is displayed, not summarised away.** Each finding carries its confidence, its detector and model version, its benign explanations, and — in the forensics panel — the raw measurements including those that did not fire.
15. **Passes are shown alongside failures,** so a Low Concern outcome is a positive statement about what was checked rather than a silence the reviewer must interpret.

None of this is sufficient on its own. A deployment would need to monitor the rate at which reviewers dismiss findings with near-identical reasons, and treat a rising rate as an operational problem rather than as efficiency.

9.6 Accessibility

The console targets the WCAG 2.2 AA contrast requirements, and the validation in Section 7.4 was computational rather than visual. Status is never conveyed by colour alone: bands carry their names, chips carry icons, the active navigation item carries a rail as well as a tint, and every chart offers the table behind it. Two limitations are acknowledged: the interface has not been tested with a screen reader by a screen-reader user, and the document viewer is inherently visual — an investigator who cannot see the page render cannot verify a region-level finding against the pixels, and the platform offers no alternative representation of that evidence.

10 Conclusion and Future Work

10.1 Conclusion

This project set out to build a platform that examines insurance claim documents for authenticity and consistency, presents every finding with its evidence and its uncertainty, ranks documents for human attention, and records the human decision that follows. All eleven objectives are met within the scope declared in Section 2.3.

The delivered system is 18,725 lines of Python and 11,077 lines of TypeScript over a 25-table schema, verified by 174 passing tests that exercise a real database and real storage, and by a 23-check walkthrough of all twenty product requirements against a live instance. It runs a fourteen-stage pipeline that combines structural, metadata, pixel-level and typographic forensics, schema-driven recognition and extraction, twenty deterministic consistency rules, cross-document comparison under pairing policies, a declared synthetic-media heuristic and six trusted-source verification routes into a single explainable evidence record, and it is deployed and operating behind TLS with its API unreachable from the internet.

The engineering contribution is the assembly rather than any individual detector: no system reviewed in Section 3 brings document-level measurement, field-level extraction, claim-level comparison and external verification into one record, ranks it for human attention, and makes the human adjudication a first-class audited part of that record.

The contribution this report would rather be judged on, however, is the discipline the system imposes on itself. Several of the most consequential decisions in the implementation were decisions *not* to claim something: measuring resampling periodicity and JPEG lattice energy and then declining to raise them on printed text; running the pixel suite over embedded rasters rather than over vector page renders and recording “not applicable” where a number could have been produced; requiring two independent detectors to agree on a region before naming it as tampering; capping the synthetic-media category below every other category because the bundled provider is a heuristic; refusing a claim approval in the API rather than warning about it in the interface. Each of those made the product quieter and each made it more honest. In a domain where the output of a measurement is used to decide whether a person's medical bill is paid, that is the correct trade — and it is a trade that has to be made in the code, because it cannot be retrofitted by documentation.

10.2 Future work

Seven directions, in the order in which they should be taken.

16. **Empirical evaluation on real claim traffic.** The first and most important. A labelled corpus of genuine claim documents with adjudicated outcomes would permit detection and false-positive rates, per-detector precision, and — critically — recalibration of the confidence functions against reality rather than against the measurement. Every threshold in Appendix A is currently justified by reasoning and by the generated corpus, not by field data.

17. **A trained synthetic-media detector behind the existing seam.** The interface, the labelling and the failure handling are implemented and tested; what is missing is a model and the corpus to evaluate it. Document images are a distinct domain from natural photographs, so a general-purpose detector should not be assumed to transfer.
18. **Live registry integration.** The production HTTP verification provider is implemented; connecting it to real policy, provider-accreditation, identity and vehicle registries is a matter of commercial access, accreditation and a data-protection assessment rather than of engineering.
19. **Learning from adjudications.** Every confirm and dismiss is stored with a written reason. That is a growing, human-labelled dataset about the platform's own signals, and the obvious use is weight calibration — a finding code dismissed by investigators in the overwhelming majority of cases is mis-weighted. The mechanism must be advisory, surfacing a proposed weight change to an administrator, rather than self-adjusting: a model that silently retunes itself against reviewer behaviour would launder automation bias into the scoring model.
20. **Complementary claim-level analytics.** Section 3.3 argued that behavioural and network fraud scoring is complementary rather than competing. Presenting a claim-level signal beside the document-level evidence, clearly distinguished as a different kind of claim about the world, would cover organised fraud that document examination cannot reach.
21. **Broader document coverage.** More document types, and non-English documents — which need different recognition models, different label synonym sets, and different date, name and currency conventions, not merely translation.
22. **Operational hardening for a real deployment.** Encryption at rest, a secrets manager in place of environment variables, database backup and restore rehearsal, horizontal worker scaling under load, and monitoring of the reviewer behaviour described at the end of Section 9.5.

11 References

Amerini, I., Ballan, L., Caldelli, R., Del Bimbo, A. and Serra, G. (2011) ‘A SIFT-based forensic method for copy–move attack detection and transformation recovery’, *IEEE Transactions on Information Forensics and Security*, 6(3), pp. 1099–1110.

Biryukov, A., Dinu, D. and Khovratovich, D. (2016) ‘Argon2: New generation of memory-hard functions for password hashing and other applications’, in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. Saarbrücken: IEEE, pp. 292–302.

Cohen, J. (1988) *Statistical power analysis for the behavioral sciences*. 2nd edn. Hillsdale, NJ: Lawrence Erlbaum Associates.

European Parliament and Council of the European Union (2016) *Regulation (EU) 2016/679 of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)*. Official Journal of the European Union, L 119, pp. 1–88.

Farid, H. (2009) ‘Image forgery detection’, *IEEE Signal Processing Magazine*, 26(2), pp. 16–25.

Farid, H. (2016) *Photo forensics*. Cambridge, MA: MIT Press.

Fridrich, J., Soukal, D. and Lukáš, J. (2003) ‘Detection of copy–move forgery in digital images’, in *Proceedings of the Digital Forensic Research Workshop (DFRWS)*. Cleveland, OH.

International Organization for Standardization (2009) *ISO 3779:2009 Road vehicles — Vehicle identification number (VIN) — Content and structure*. Geneva: ISO.

International Organization for Standardization (2020) *ISO 13616-1:2020 Financial services — International bank account number (IBAN) — Part 1: Structure of the IBAN*. Geneva: ISO.

Jones, M., Bradley, J. and Sakimura, N. (2015) *JSON Web Token (JWT)*, RFC 7519. Internet Engineering Task Force.

Krawetz, N. (2007) *A picture's worth: digital image analysis and forensics*. Hacker Factor Solutions white paper.

Lukáš, J., Fridrich, J. and Goljan, M. (2006) ‘Digital camera identification from sensor pattern noise’, *IEEE Transactions on Information Forensics and Security*, 1(2), pp. 205–214.

Machado, G.M., Oliveira, M.M. and Fernandes, L.A.F. (2009) ‘A physiologically-based model for simulation of color vision deficiency’, *IEEE Transactions on Visualization and Computer Graphics*, 15(6), pp. 1291–1298.

Open Web Application Security Project (2021) *OWASP Application Security Verification Standard 4.0.3*. OWASP Foundation.

Ottosson, B. (2020) *A perceptual color space for image processing (Oklab)*. Available at: https://bottosson.github.io/posts/oklab/.

Pearl, J. (1988) *Probabilistic reasoning in intelligent systems: networks of plausible inference*. San Mateo, CA: Morgan Kaufmann.

Popescu, A.C. and Farid, H. (2005) ‘Exposing digital forgeries by detecting traces of resampling’, *IEEE Transactions on Signal Processing*, 53(2), pp. 758–767.

Rublee, E., Rabaud, V., Konolige, K. and Bradski, G. (2011) ‘ORB: An efficient alternative to SIFT or SURF’, in *2011 International Conference on Computer Vision (ICCV)*. Barcelona: IEEE, pp. 2564–2571.

Smith, R. (2007) ‘An overview of the Tesseract OCR engine’, in *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. Curitiba: IEEE, pp. 629–633.

World Wide Web Consortium (2023) *Web Content Accessibility Guidelines (WCAG) 2.2*, W3C Recommendation, 5 October 2023.

Appendix A Detector Catalogue

Every finding code the platform can emit, with the property it measures and its seeded severity and confidence treatment. Confidence values are calibrated per detector against the measurement itself; **they are not probabilities of fraud**. The repository's `docs/DETECTION.md` carries, in addition, the benign explanations an investigator should rule out for each code.

A.1 Metadata provenance — PDF

Detector `pdf-structure-forensics` v1.5.0. Producer and creator strings are classified into nine software families; three are treated as high-concern (raster editor, design tool, online editor) and two as medium (PDF manipulation, PDF editor).

Table 22. PDF metadata findings.

Code	Measured	Severity / confidence
`PDF_MODDATE_BEFORE_CREATION`	Modification date precedes creation by more than 60 s	high / 0.88
`PDF_EDITED_AFTER_CREATION`	More than one day between creation and last save	medium ≥7 d else low / 0.45–0.80 by elapsed days
`PDF_FUTURE_TIMESTAMP`	A document timestamp later than server time + 1 day	medium / 0.70
`PDF_IMAGE_EDITOR_PRODUCER`	Producer or creator is a raster or design editor	high / 0.78
`PDF_MANIPULATION_TOOLING`	Producer or creator is a merge, split or re-save tool	low / 0.50
`PDF_PRODUCER_CREATOR_MISMATCH`	Creator and producer belong to different software families	low / 0.55
`PDF_METADATA_ABSENT`	Document information dictionary is empty	low / 0.45
`PDF_PRODUCER_ABSENT`	Neither producer nor creator is present	low / 0.42

A.2 Metadata provenance — images

Detector `image-metadata-forensics` v1.2.0. Eleven editor markers are recognised in the EXIF software tag; eight camera keys define capture provenance.

Table 23. Image metadata findings.

Code	Measured	Severity / confidence
`IMG_EDITOR_SOFTWARE_TAG`	EXIF software tag names an image editor	high / 0.80
`IMG_EXIF_TIMESTAMP_DRIFT`	Modification timestamp more than an hour after capture	medium / 0.62
`IMG_EXIF_FUTURE_TIMESTAMP`	An EXIF timestamp later than now	medium / 0.65
`IMG_EXIF_STRIPPED`	No EXIF at all on a photographic format	low / 0.45

A.3 Document forensics — pixel suite

Detector `image-forensics-suite` v1.6.0. All metrics are computed over **content blocks only** — blocks whose local standard deviation clears a floor — because a scanned document is mostly blank paper and including it makes every text block look like an outlier. Region significance is established by a sliding-window two-sample test against the remainder of the page, requiring both statistical strength and a minimum effect size.

Table 24. Pixel-suite findings and their gating thresholds.

Code	Measured	Gate	Confidence
`IMG_ELA_LOCAL_ANOMALY`	Region whose response to controlled re-compression differs from the rest of the page, in either direction	z ≥ 7.0, effect ≥ 0.40; top 3 regions	0.42–0.90, damped ×0.8 on a lossless render
`IMG_NOISE_INCONSISTENCY`	Local noise floor from a median-filtered residual differing from the page	z ≥ 9.5, effect ≥ 0.60; top 2	0.40–0.82, ceiling medium
`IMG_COPY_MOVE_REGION`	ORB self-matching with offsets clustered at 8 px	≥ 22 matches sharing one displacement, ≥ 0.6 % of page area; top 2	0.48–0.88
`IMG_COMPRESSION_INCONSISTENCY`	JPEG-ghost quality map: a region whose last compression quality differs from the page	z ≥ 7.0, effect ≥ 0.45; JPEG sources only; top 2	0.50–0.90
`IMG_LOCAL_BLOCKINESS_ANOMALY`	Gradient energy on and off the 8 px JPEG lattice	robust z ≥ 4.0; **suppressed when text dominance ≥ 0.35**	0.50–0.86
`IMG_RESAMPLING_ARTIFACTS`	Periodic peak in the second-derivative spectrum per axis	peak ratio ≥ 20.0; **suppressed when text dominance ≥ 0.35**	0.40–0.72, ceiling medium
`IMG_LOW_JPEG_QUALITY`	Estimated JPEG quality below 62	—	low / 0.60 — flags that other detectors' coverage is degraded
`IMG_NONSTANDARD_QUANT_TABLE`	Luma quantisation table off the standard IJG scaling curve	ratio σ ≥ 0.08	low / 0.50

Also measured and retained as context but never raised as signals: the JPEG lattice energy ratio and a per-block high-frequency sparsity measure, both flagged `used_as_signal: false` with the reason recorded. Artefact produced: a colour-mapped error-level-analysis heatmap per analysed page, stored in the artifacts scope and selectable as a viewer layer.

A.4 Document forensics — PDF structure

Table 25. PDF structural findings.

Code	Measured	Severity / confidence
`PDF_INCREMENTAL_UPDATE`	Cross-reference generations: end-of-file and startxref counts, `/Prev` references	medium if ≥2 increments else low / 0.55–0.90
`PDF_TRAILING_DATA`	More than 1 KiB of bytes after the final end-of-file marker	medium / 0.60
`PDF_TEXT_FONT_MIX_IN_LINE`	**Exact**: characters on one baseline-clustered line drawn from different font resources	by confidence 0.50–0.92; highest source reliability in the correlation layer

Code	Measured	Severity / confidence
PDF_MIXED_FONT_EMBEDDING	A page mixing embedded and non-embedded fonts	medium / 0.62
PDF_FONT_DIVERSITY	Six or more distinct base fonts on one page	low / 0.40–0.70
PDF_MIXED_IMAGE_COMPRESSION	Two or more embedded images using different filter chains	medium / 0.60
PDF_MASKED_IMAGE_OVERLAY	Small embedded images carrying a soft mask	medium / 0.58
PDF_MIXED_BIT_DEPTH	More than one distinct bits-per-component among embedded images	low / 0.45
PDF_CONTENT_ANNOTATIONS	FreeText, square, highlight, stamp, ink or redaction annotations drawn over the page	medium for FreeText / redact / stamp else low / 0.72

A.5 Typography and layout

Detector typography-layout-forensics v1.4.0. Per recognised line of at least three words; numeric tokens are ranked first and receive a small confidence premium; at most four findings per code per page. Each emits a suspect region for the correlation layer.

Table 26. Typography findings.

Code	Measured	Threshold	Ceiling
TXT_BASELINE_ANOMALY	Word baseline against the line median, normalised by glyph height	deviation > 0.34, line ≥ 5 words	medium
TXT_GLYPH_SIZE_ANOMALY	Word height against the line median	ratio > 1.42 or < 0.66	medium
TXT_STROKE_WEIGHT_ANOMALY	Mean stroke half-width via Otsu threshold and a distance transform	relative deviation > 0.42	high
TXT_LOW_CONFIDENCE_ISLAND	A word recognised below 0.58 inside a line above 0.82	as stated	medium

A.6 Tampering correlation

Detector tampering-correlation-engine v1.5.0. A region is reported only when a high-reliability source fires or two independent sources overlap on the same area, and the combined confidence clears 0.45.

```
combined = clamp( 1 − Π ( 1 − confidence_i × reliability_i ), 0, 0.93 )
```

Table 27. Source reliabilities used in the noisy-OR combination. The first three are high-reliability sources that may raise a region alone.

Source	Reliability	Source	Reliability
Exact PDF font mix in a line	**0.90**	Embedded image mismatch	0.68
Copy-move region	**0.85**	Stroke-weight anomaly	0.66
PDF content annotation	**0.75**	Noise inconsistency	0.60
Error-level analysis anomaly	0.72	Baseline anomaly	0.58
Local compression quality	0.70	Glyph-size anomaly	0.54
		Low-confidence island	0.48

Thirteen tamper types are produced, named by intersecting the region with the extracted fields: `TAMPER_EDITED_AMOUNT`, `TAMPER_MODIFIED_DATE`, `TAMPER_ALTERED_IDENTIFIER`, `TAMPER_REPLACED_NAME`, `TAMPER_REPLACED_TEXT`, `TAMPER_EDITED_VALUE`, `TAMPER_COPY_PASTE_REGION`, `TAMPER_OVERLAY_ANNOTATION`, `TAMPER_IMAGE_REPLACEMENT`, `TAMPER_ADDED_TEXT`, `TAMPER_FONT_SUBSTITUTION`, `TAMPER_SIGNATURE_OR_STAMP`, and `TAMPER_REGION_ANOMALY` where the region covers no extracted value.

A.7 Synthetic-media indicators

Provider `veridoc-demo-heuristic` v0.4.0, declared as a demonstration provider. Eight measured features with published weights summing to 1.0; the likelihood is their weighted sum capped at 0.97, damped ×0.82 when the source is a lossless page render.

Table 28. Synthetic-media indicators, their measurement and their published weights.

Indicator	Measured quantity	Weight
Spectral roll-off	Share of Fourier energy beyond 0.62 of the maximum radius, inverted	0.19
Noise flatness	Coefficient of variation of block noise in a median-filter residual, inverted	0.17
Provenance absent	Whether camera EXIF is present (0.25 for a lossless render)	0.13
Channel coupling	Mean absolute correlation of per-channel Laplacians	0.12
Text-render irregularity	Combined variation of recognition confidence and glyph height	0.12
Gradient kurtosis	Fourth moment of the normalised Sobel magnitude, descending	0.11
Variance entropy	Shannon entropy of the block-variance histogram	0.09
Palette continuity	Unique colours per pixel on a downsample	0.07

Table 29. Likelihood bands. There is no code path that can emit a verdict.

Likelihood	Band	Wording shown

Likelihood	Band	Wording shown
0.50 – 0.68	`possible_indicators`	Potential synthetic-media indicators detected
≥ 0.68	`notable_indicators`	Notable synthetic-media indicators detected

A.8 File security

Table 30. Security findings.

Code	Measured	Severity / confidence
`SEC_MALWARE_SIGNATURE`	Scanner verdict `infected`	**critical / 0.97** — quarantines the document and halts stages 3–13 without deleting it
`SEC_ACTIVE_CONTENT`	Scanner verdict `suspicious`: active-content patterns, obfuscated script, ZIP polyglot or a PE header	high / 0.72
`PDF_EMBEDDED_JAVASCRIPT`	JavaScript name tree or script in an open action	high / 0.85
`PDF_EMBEDDED_FILES`	Embedded-file name tree present	medium / 0.70

A.9 Consistency and cross-document

The twenty consistency rules are listed by family in Section 6.5 with their seeded severities; the seventeen cross-document comparable fields and their three pairing policies are in Section 6.6. Two further consistency-category codes are emitted by the pipeline itself: `DOC_DUPLICATE_CONTENT_HASH` (detector `ingest-integrity` v1.0.0 — a byte-identical file already on record, attributed to the later submission) and `CLS_UNRECOGNISED_DOCUMENT_TYPE` (the classifier scored below its floor). Cross-document codes take the form `XDOC_<FIELD>_MISMATCH` for document-to-document disagreement and `XDOC_CLAIM_<FIELD>_MISMATCH` for disagreement with the claim record.

Appendix B Risk Weights and Ceilings

The risk model is data, not code: 99 rows in the `detection_rules` table carry a severity and a weight per finding code, and an administrator can retune any of them with the change audited. The values below are the seeded defaults for weights version 2026.09.1.

```
        contribution(signal) = weight(code) × confidence × severity_multiplier
        category_score       = cap × (1 − exp(−1.3 × Σ contributions / cap))
        score                = round(100 × Σ category_scores / 72) − mitigation

        severity multipliers   info 0.35 · low 0.60 · medium 1.00 · high 1.35 ·
critical 1.60
        saturation steepness   1.30
        calibration divisor    72.0
        maximum mitigation     6.0  (2.0 per corroborating external match;
                                     suppressed entirely if any high or critical signal
exists)
        bands                  Low Concern < 40 · Review Recommended 40–74 · High
Concern ≥ 75
```

Table 31. Category ceilings, rule counts and the fallback weight applied when a finding code has no row of its own.

Category	Ceiling	Seeded rules	Fallback weight
External verification	70	18	26.0
Tampering indicators	58	13	22.0
Internal consistency	50	2 + 20 engine rules	14.0
Cross-document agreement	50	9	20.0
File security	40	4	18.0
Document forensics	34	21	11.0
Metadata provenance	24	12	10.0
Synthetic-media indicators	20	1	12.0

Table 32. Highest and lowest seeded weights, illustrating the model's ordering of evidence.

Finding code	Weight	Rationale
`SEC_MALWARE_SIGNATURE`	30.0	A malicious file is decisive about the submission.
`VERIFY_IDENTITY_LOOKUP_MISMATCH`	30.0	The issuing authority's record contradicts the document.
`VERIFY_POLICY_LOOKUP_MISMATCH`	28.0	The insurer's own record contradicts the document.
`VERIFY_INVOICE_LOOKUP_MISMATCH`	28.0	The issuer's billing record contradicts the amount.
`TAMPER_EDITED_AMOUNT`	24.0	Independent detectors agree on a region covering a money field.
`XDOC_TOTAL_MISMATCH`	24.0	Two documents in one bundle state different totals for one invoice.
`XDOC_CLAIM_TOTAL_MISMATCH`	23.0	The document contradicts the claim record.

Finding code	Weight	Rationale
PDF_TEXT_FONT_MIX_IN_LINE	19.0	An exact structural fact, not a statistical inference.
LINE_ITEM_SUM_MISMATCH	18.0	Deterministic arithmetic on extracted values.
PDF_MODDATE_BEFORE_CREATION	16.0	Impossible ordering, but trivially forgeable and often a clock error.
AIGC_SYNTHETIC_INDICATORS	13.0	Deliberately modest: the bundled provider is a heuristic.
IMG_LOW_JPEG_QUALITY	4.0	Informative about coverage rather than about the document.

Appendix C API Endpoint Reference

58 endpoints across 12 routers, all under `/api`. Interactive documentation is served at `/docs` and the schema at `/openapi.json`. Every mutating endpoint declares the permission it requires and enforces it server-side. Errors are uniform: `{"error": {"code", "message", "details", "retryable"}}`.

Table 33. System and authentication endpoints.

Method	Path	Permission	Purpose
GET	`/health`	—	Liveness probe.
GET	`/ready`	—	Readiness: database, storage, queue, versions, environment.
GET	`/system/capabilities`	—	Upload limits, provider availability, queue mode, versions.
POST	`/auth/login`	—	Credentials to access and refresh tokens; rate limited per IP and per account.
POST	`/auth/refresh`	—	Rotate both tokens; an access token is rejected.
POST	`/auth/logout`	session	Record the sign-out in the audit trail.
GET	`/auth/me`	session	Current user with the effective permission list.
POST	`/auth/change-password`	session	Change own password under the strength policy.
GET	`/auth/roles`	session	Role catalogue with permissions.

Table 34. Claim, document and dashboard endpoints.

Method	Path	Permission	Purpose
GET	`/dashboard`	`dashboard:read`	Queue, band, type, finding and verification aggregates over a window.
POST	`/claims`	`claim:create`	Create a claim.
GET	`/claims`	`claim:read`	Paged list with status, band, assignee and search filters.
GET	`/claims/{id}`	`claim:read`	Claim with documents, findings and investigations.
PATCH	`/claims/{id}`	`claim:update`	Update claim fields; reassignment additionally requires `investigation:assign`.
POST	`/claims/{id}/decision`	`claim:decide`	Record the human decision. **409 when approving with any high or critical finding still open.**
POST	`/documents`	`document:upload`	Validated multipart upload with hashing, duplicate warning and optional analysis.

Method	Path	Permission	Purpose
GET	`/documents`	`document:read`	Paged queue with band, status, type and verdict filters.
GET	`/documents/{id}`	`document:read`	The full investigation payload; audits the view.
GET	`/documents/{id}/findings`	`document:read`	Unified findings with the provenance legend.
GET	`/documents/{id}/analysis`	`document:read`	Every analysis run with its stage timeline.
POST	`/documents/{id}/analyze`	`document:analyze`	Queue or re-queue the pipeline.
POST	`/documents/{id}/classification`	`document:analyze`	Override the document type, recording who and why.
POST	`/documents/{id}/review`	per action	Record a human review action.
POST	`/documents/{id}/reassess`	`document:analyze`	Recompute the score after review.
POST	`/documents/{id}/links`	`document:download`	Mint short-lived signed links.
GET	`/documents/{id}/file`	signed or `document:download`	Stream the original; audits the download.
GET	`/documents/{id}/pages/{n}/render`	signed or `document:download`	Stream a page render or a forensic layer.
DELETE	`/documents/{id}`	`document:delete`	Soft delete with a reason.

The review endpoint accepts six actions, each mapped to its own permission: `confirm_finding` (`finding:confirm`), `dismiss_finding` (`finding:dismiss`), `request_information` (`finding:request_info`), `mark_document_verified` and `reject_document` (`document:verify`), and `escalate_investigation` (`investigation:escalate`). All except `request_information` require a written reason of at least ten characters, enforced server-side.

Table 35. Verification, investigation, report, audit, user and administration endpoints.

Method	Path	Permission	Purpose
GET	`/verification/providers`	`verification:read`	Providers with mode and demo flag.
GET	`/verification/requests`	`verification:read`	Paged verification history.
POST	`/verification/documents/{id}/run`	`verification:run`	Run checks on demand.
GET	`/investigations`	`investigation:read`	Paged case list.
POST	`/investigations`	`investigation:escalate`	Open a case.
GET	`/investigations/{id}`	`investigation:read`	Case detail with notes.
PATCH	`/investigations/{id}`	`investigation:note`	Update a case; closing requires `investigation:close`.
POST	`/investigations/{id}/notes`	`investigation:note`	Add a note.
POST	`/reports`	`report:generate`	Generate the PDF evidence report.
GET	`/reports`	`report:read`	Paged report register.
GET	`/reports/{id}`	`report:read`	Report metadata with both hashes.
GET	`/reports/{id}/export`	`report:read`	Download the PDF.
GET	`/audit-logs`	`audit:read`	Paged, filtered audit trail.

Method	Path	Permission	Purpose
GET	`/audit-logs/actions`	`audit:read`	Known audit action codes.
GET	`/audit-logs/chain/verify`	`audit:read`	Recompute the hash chain and name the first divergence.
GET	`/users`	`user:read`	Paged user list.
GET	`/users/{id}`	`user:read`	User detail.
POST	`/users`	`user:write`	Create a user.
PATCH	`/users/{id}`	`user:write`	Update role or activation.
GET	`/users/investigators`	`claim:read`	Investigators available for assignment.
GET	`/users/workload`	`investigation:read`	Open-case load per investigator.
GET	`/rules`	`rule:read`	Detection rule catalogue with weights.
GET	`/rules/catalogue`	`rule:read`	Built-in definitions and the risk weight model.
PATCH	`/rules/{code}`	`rule:write`	Retune a rule; audited.
GET	`/document-types`	`rule:read`	Schemas, required fields, synonyms and verification routes.
GET	`/providers`	`provider:read`	Provider configuration — credential variable **names** only.
GET	`/providers/runtime`	`provider:read`	Live provider availability.
PATCH	`/providers/{kind}/{key}`	`provider:write`	Activate or configure a provider.
GET	`/settings`	`settings:read`	System settings.
PUT	`/settings/{key}`	`settings:write`	Update a setting; risk thresholds are validated.

Appendix D Test Inventory

155 test functions across 10 files, expanding with parametrisation to 174 executed tests, all passing in 157.78 s. The suite runs against a real PostgreSQL database and real storage; only external providers are substituted.

Table 36. Notable assertions per test file, emphasising the negative and wording assertions.

File	Representative assertions
`test_auth_rbac.py`	Sign-in issues typed tokens; failure wording is identical for an unknown account and a wrong password; an access token cannot be replayed as a refresh token; refresh rotates both tokens; the permission matrix matches per role; each role is denied the endpoints it must not reach.
`test_upload_security.py`	A PE executable is rejected; an extension-spoofed file is caught by content sniffing; corrupt, empty and oversized uploads are rejected; a declared-MIME mismatch warns without rejecting; **EICAR is quarantined with the file preserved**; the stored SHA-256 equals the uploaded bytes; a duplicate is reported and both copies kept; a reviewer cannot upload.
`test_extraction.py`	Money, date, identifier and name normalisation; `is_same_person` distinguishes a dependant from the policyholder; the text layer is preferred for digital PDFs; the classifier identifies an invoice and declines on noise; fields carry boxes and confidences; requiredness is per-type; **a heading is not read as a label**.
`test_rules_engine.py`	Each rule fires on its own violation and passes on a clean document; a rule returns not-applicable rather than passing when inputs are absent; a disabled rule does not run; severity and weight overrides take effect; document-type scoping holds; the catalogue is complete; **a deliberately broken rule is isolated** without failing the stage.
`test_crossdoc_verification.py`	Invoice and claim form totals mismatch; **unrelated invoices are not compared on total**; **a dependant's date of birth is not compared with the policyholder's**; identical values yield no findings; a single document yields nothing; all six demo providers are labelled demo and carry the disclaimer even on a match; a lapsed policy outside cover grades as mismatch; an unknown policy grades as not found; a peripheral failure grades as partial; the unconfigured production provider raises.
`test_risk_and_aigc.py`	No signals yields Low Concern; the score is monotonic in severity and count; a category saturates towards its ceiling; dismissed signals are excluded and reported; confirmed signals are promoted; mitigation is suppressed when a severe finding exists; configured thresholds are honoured; **the explanation never asserts a probability of fraud**; **synthetic-media output is always a band**; the heuristic is deterministic; a clean render yields no significant indicators.
`test_pipeline_e2e.py`	A clean invoice completes every stage; a tampered invoice produces the expected evidence; findings satisfy the full evidence contract; **a recognition failure degrades only its stage and preserves the document**; a synthetic-media failure is isolated; **re-analysis preserves human review decisions**; renders and forensic layers are stored; a vector PDF skips the inapplicable pixel metrics; the run records versions and the provider snapshot.

File	Representative assertions
`test_api_workflow.py`	Upload wording; list filtering; every evidence panel present in the payload; findings filterable with provenance labels; signed links honoured and **a tampered signature rejected**; a missing render layer reports what is available; **a reason is required to confirm**; confirm then reassess changes the score; a reviewer cannot escalate but an investigator can; **approval is blocked while high findings are unreviewed**; verification on demand; notes; report generated and downloadable; the audit trail records the workflow and verifies.
`test_audit_and_admin.py`	Entries chain to their predecessor; **rewriting history breaks the chain and the divergence is named**; sensitive values are redacted before storage; the log formatter redacts personal data; rules are retunable with an audit entry; **providers never expose credential values**; activating a provider deactivates its peers; risk thresholds are validated; user creation and role change are audited.
`test_storage_and_errors.py`	Traversal keys are rejected; originals and artifacts are isolated; **originals are written read-only**; artifacts purge without touching originals; a signed resource round-trips and is bound to resource, purpose and user, and expires; token types are enforced; error payloads carry a code and retryability; `ProviderUnavailable` names the provider; security headers are present; **document bytes are never cacheable**.

Appendix E Data Dictionary

25 tables, created by a single Alembic revision generated from the models. Conventions: UUID string primary keys, `created_at` and `updated_at` on every table, `deleted_at` where soft deletion applies, and JSONB for structured evidence so that a new detector's evidence shape does not require a migration.

Table 37. Table reference. Key columns only; every table also carries its identifier and timestamps.

Table	Purpose and key columns
`roles`	Role catalogue. `name` unique, `label`, `permissions` (JSONB), `is_system`.
`users`	`email` unique, `full_name`, `password_hash`, `role_name`, `job_title`, `team`, `is_active`, `must_change_password`, `failed_login_count`, `locked_until`, `last_login_at`, soft delete.
`claims`	`claim_number` unique, customer particulars, `policy_number`, `claim_type`, `incident_date`, `claimed_amount`, `currency`, `status`, `risk_score`, `risk_band`, `decision`, `decision_reason`, `decided_at`, `decided_by_id`, `assigned_investigator_id`.
`documents`	`document_ref` unique; immutable ingest facts `original_filename`, `storage_key`, `artifact_prefix`, `mime_type`, `declared_mime_type`, `file_size`, **`sha256`**, `page_count`, `source`; pipeline state `processing_status`, `malware_status`, `classification_type/confidence`, `risk_score`, `risk_band`; human verdict `verification_state`, `verified_by_id`, `verified_at`; `retention_expires_at`; soft delete.
`document_pages`	Unique per document and page. `width`, `height`, `render_key`, `render_dpi`, `has_text_layer`, `text_char_count`, `artifacts` (thumbnail and forensic layer keys).
`document_metadata`	`group_name` (pdf_info, xmp, exif, image_info, file, structure), `key`, `value`, `is_anomalous`, `note`, `raw`.
`ocr_results`	Unique per document and page. `provider`, `provider_version`, `status`, `raw_text`, `mean_confidence`, `word_count`, `words` (with boxes), `duration_ms`.
`extracted_fields`	`field_key`, `field_label`, `value_raw`, `value_normalized`, `value_type`, `confidence`, `page`, **`bbox`**, `source`, `is_required`, `group_name`, `line_index`.
`analysis_runs`	`status`, `queue_job_id`, `queue_mode`, `pipeline_version`, `rules_version`, **`stages`** (the 14-stage timeline), `provider_snapshot`, timings, `findings_count`, `reprocess_of_id`.
`risk_assessments`	`score`, `band`, `weights_version`, `signals`, `category_scores`, `top_reasons`, `explanation`, `interpretation_note`, `coverage`, `is_superseded` — prior assessments are flagged, never deleted.
`forensic_findings`	Shared finding contract plus `signal_group`; holds the metadata, forensics and security categories.
`aigc_findings`	Shared contract plus `provider_key`, `provider_label`, **`is_demo_provider`**, `indicator_scores`, `likelihood_band`.
`tampering_findings`	Shared contract plus `tamper_type`, `affected_field`, `affected_value`, `overlay_key`.
`consistency_findings`	Shared contract plus `rule_code`, `fields_involved`, `expected`, `actual`.

Table	Purpose and key columns
cross_document_findings	Shared contract plus mandatory claim_id, counterpart_document_id, field_key, value_a, value_b, similarity.
verification_requests	provider_key, provider_label, **`provider_mode`**, request_type, request_payload, status, trigger.
verification_results	outcome, summary, checks, raw_response, latency_ms, reference_id, **`is_demo_data`**, **`data_source_disclaimer`**, error.
investigations	case_number unique, status, priority, assigned_to_id, opened_by_id, opened_at, closed_at, resolution, risk_score_at_open, context.
investigation_notes	author_id, body, note_type, is_internal, attachments.
review_actions	Append-only human decisions: finding_kind, finding_id, action, **`reason`**, actor_id, actor_role, previous_status, new_status.
reports	report_ref unique, report_type, format, storage_key, file_size, **`sha256`**, **`document_sha256`**, pipeline_version, rules_version, snapshot.
audit_logs	**`seq`** (gap-free database identity), actor_id/email/role, action, resource_type, resource_id, summary, ip_address, user_agent, request_id, before_state, after_state, **`prev_hash`**, **`entry_hash`**.
detection_rules	code unique, category, default_severity, **`weight`**, enabled, applies_to, params, version, is_system, updated_by_id — read at run time by the consistency and risk engines.
provider_configs	Unique per kind and key. is_active, **`is_demo`**, **`credential_env_keys`** (variable *names* only — never secrets), settings_json, model_name, model_version, health_status.
system_settings	key unique, label, value (JSONB), description. Seeded keys: risk_thresholds, retention, review_policy, analysis_policy, privacy_policy.

Appendix F Configuration Reference

All configuration is environment-driven and validated at start-up. The settings that change behaviour most are listed below with their shipped defaults; the active recognition, synthetic-media and verification providers can additionally be switched at run time from the console, in which case the database setting overrides the environment default.

Table 38. Principal configuration variables.

Variable	Default	Effect
`APP_ENV`	`development`	One of development, staging, production, test; production enables HSTS.
`DATABASE_URL`	local PostgreSQL	Connection string; pool size and overflow are separately configurable.
`JWT_SECRET`	*must be set*	Signing key. The production compose file fails the deploy loudly if it is absent.
`ACCESS_TOKEN_TTL_MINUTES`	`45`	Access-token lifetime; refresh is 7 days.
`PASSWORD_HASH_SCHEME`	`argon2`	Argon2id, with bcrypt accepted for legacy verification.
`SIGNED_URL_TTL_SECONDS`	`300`	Lifetime of an HMAC-signed document link.
`RATE_LIMIT_LOGIN`	`10/minute`	Per-IP sign-in limit; a per-account limit applies in addition.
`MAX_UPLOAD_MB`	`25`	Hard ceiling enforced while streaming.
`MAX_PAGE_COUNT`	`60`	Rejected above this page count.
`RENDER_DPI`	`150`	Page rasterisation resolution; the coordinate space for every region.
`STORAGE_PROVIDER`	`local`	`local` filesystem or `s3` for any S3-compatible store.
`S3_SERVER_SIDE_ENCRYPTION`	unset	Enables encryption at rest without a code change.
`QUEUE_MODE`	`redis`	`redis` uses RQ workers; `inline` analyses in-process with no worker.
`OCR_PROVIDER`	`hybrid`	`hybrid`, `tesseract`, `pdf_text` or `demo_fixture`.
`AIGC_PROVIDER`	`demo_heuristic`	`demo_heuristic` or `remote_http` with a URL and key.
`VERIFICATION_PROVIDER`	`demo`	`demo` or `remote_http` with a URL and key.
`MALWARE_SCANNER`	`heuristic`	Static checks, or `clamav` against a clamd service.
`RETENTION_DAYS`	`365`	Feeds the retention job; artifacts are purged before originals.
`PII_LOG_REDACTION`	`true`	Redaction in the log formatter; document bodies are never logged regardless.
`TRUSTED_HOSTS` / `ALLOWED_ORIGINS`	restrictive in production	Host and CORS allow-lists.
`BACKEND_API_URL`	internal service DNS	Read by the console **server-side only**; never reaches the browser.
`NEXT_PUBLIC_DEMO_MODE`	`true` in the demo	Shows the demonstration banner and the role reference on sign-in.

Appendix G Deployment Runbook

G.1 Local development

```
        cp .env.example .env                  # set JWT_SECRET at minimum
        docker compose up -d --build          # postgres, redis, minio, api,
worker, web
        docker compose exec api python -m app.db.seed --reset

        # console  http://localhost:3010     API docs  http://localhost:8088/docs
        # sign in with any demo account; the password is set by SEED_PASSWORD
```

Without Docker: PostgreSQL 14 or later plus `tesseract-ocr`, `tesseract-ocr-eng`, `poppler-utils` and `libmagic1`; then `alembic upgrade head`, `python -m app.db.seed --reset`, `uvicorn app.main:app --port 8088`, and — when `QUEUE_MODE=redis` — `python -m app.workers.worker` in a second terminal. `make help` lists every task.

G.2 Production deployment

```
        cd /root/stacks/veridoc
        docker compose up -d --build          # rebuilds and recreates only what
changed
        docker compose ps                     # all five services should report
healthy
        docker compose logs -f api worker

        docker compose up -d --scale worker=2    # horizontal analysis capacity
```

The production stack publishes no host ports and routes only the console. Secrets are supplied through an environment file readable by root alone; the required values are the application domain, the JWT signing secret and the database password, each declared so that a missing value fails the deploy rather than silently booting on a default. The API's entrypoint waits for the database, applies migrations and then execs the server; seeding runs only when explicitly requested, so a container restart can never populate a production database with demonstration claims.

G.3 Operational tasks

Table 39. Routine operations.

Task	Command or action
Apply migrations	`alembic upgrade head` (automatic in the API entrypoint)
Check for model drift	`alembic check` — fails if the models and migrations have diverged
Seed reference data only	`python -m app.db.seed --reference-only`
Reset and seed the demonstration	`python -m app.db.seed --reset` — destructive
Retention dry run	`make purge-dry` — reports what would be purged; artifacts before originals, and originals only for soft-deleted documents

Task	Command or action
Verify the audit chain	`GET /api/audit-logs/chain/verify`, or the banner at the top of the audit screen
Readiness check	`GET /api/ready` — database, storage, queue depth, worker count, versions
Retune a rule	Rules screen as an administrator, or `PATCH /api/rules/{code}` — audited either way
Switch a provider	Integrations screen, or `PATCH /api/providers/{kind}/{key}` — activating one deactivates its peers
Change band thresholds	Settings screen, or `PUT /api/settings/risk_thresholds` — validated as an increasing pair within 0–100

G.4 Failure modes and responses

Table 40. Expected failure modes and the system's behaviour.

Failure	Behaviour and response
Redis unreachable	Uploads are analysed in-process as a background task and the run records `queue_mode=inline`. No work is lost; throughput falls. Restore Redis and subsequent uploads return to the worker.
Worker not running with `QUEUE_MODE=redis`	Uploads remain `queued` indefinitely. `GET /api/ready` reports the worker count; start `python -m app.workers.worker`.
A provider is down	The dependent stage records `unavailable`, the run is `partial`, the risk assessment discloses the degraded coverage, and the document can be re-analysed once the provider returns.
Object storage unavailable	An S3 failure falls back to local storage with an error logged, so an upload is never lost.
Analysis fails unexpectedly	The run is marked failed and the document's message states that the original is preserved and can be reprocessed. Nothing is deleted.
Infected upload	The document is quarantined, not deleted: the bytes stay downloadable as evidence and a critical security finding is recorded.
Audit chain divergence	The verification endpoint reports `intact: false` and names the first divergent sequence number — treat as a security incident.

Appendix H Console Screens

The remaining screens of the deployed console, captured from the live deployment in the light theme. All twelve screens are also available in a dark theme whose steps are chosen independently rather than derived by inversion.

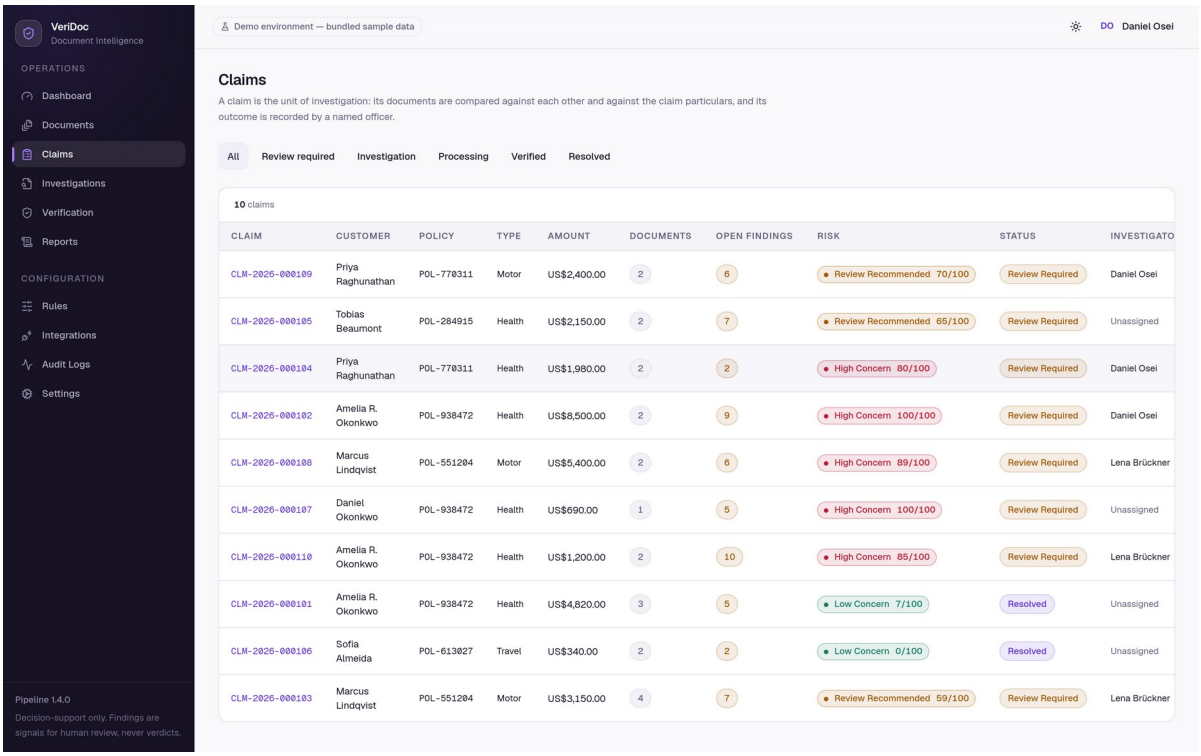


Figure 7. The claims register. A claim is the unit of investigation: its documents are compared against each other and against the claim particulars.

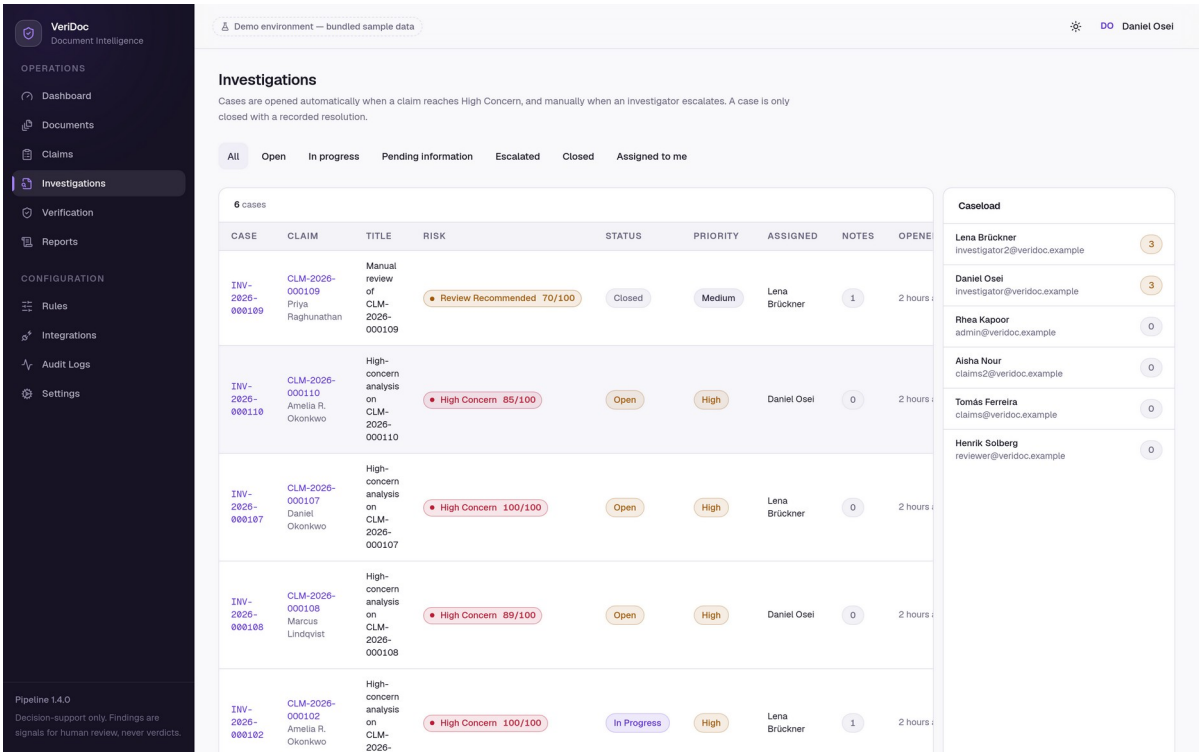


Figure 8. Open investigations with the per-investigator workload panel. A high-concern claim opens a case automatically, assigned to the least-loaded investigator.

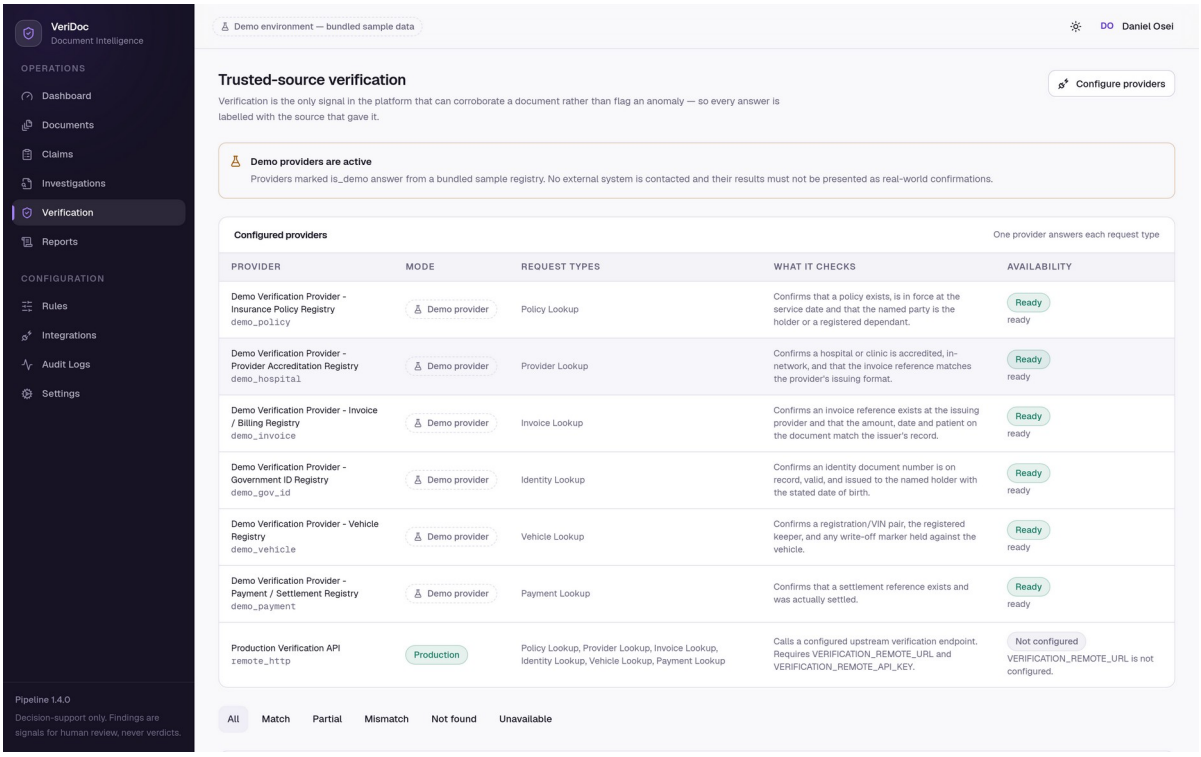


Figure 9. Trusted-source verification: configured providers with their mode, and the request history with outcome filters. Demonstration providers are badged.

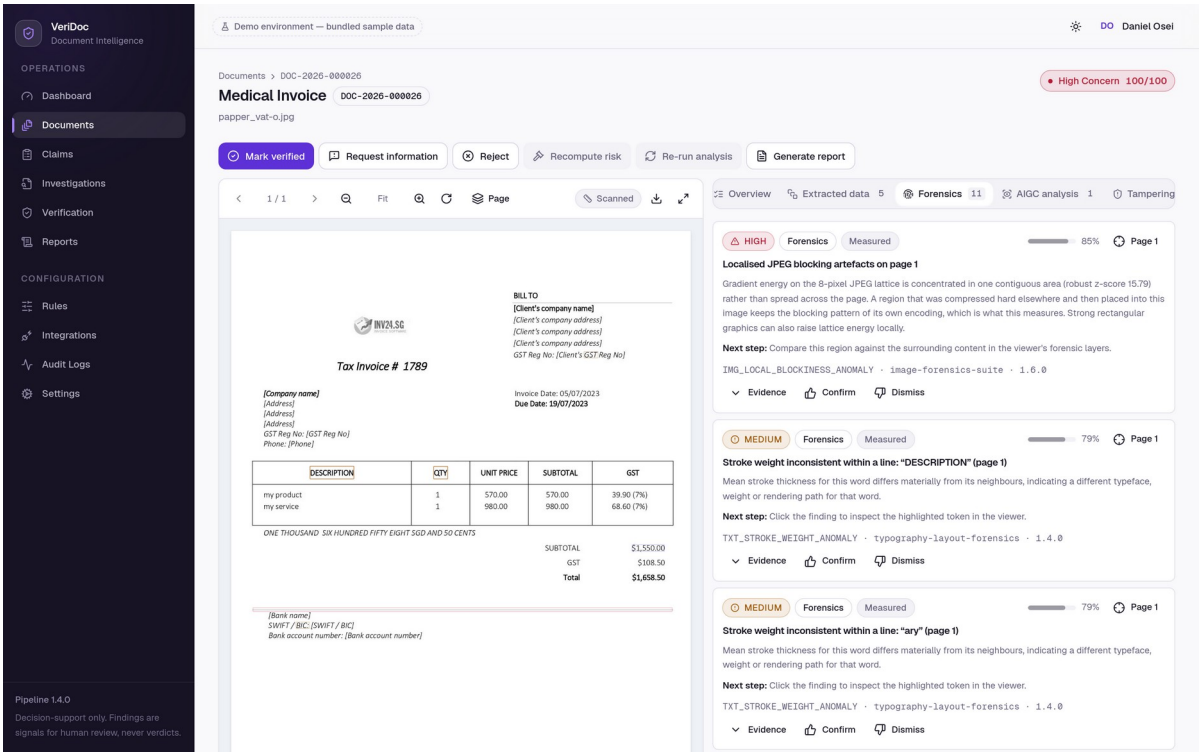


Figure 10. The forensics panel, showing findings, the full metadata table with anomalous rows annotated, and the raw measurements — including metrics that did not fire.

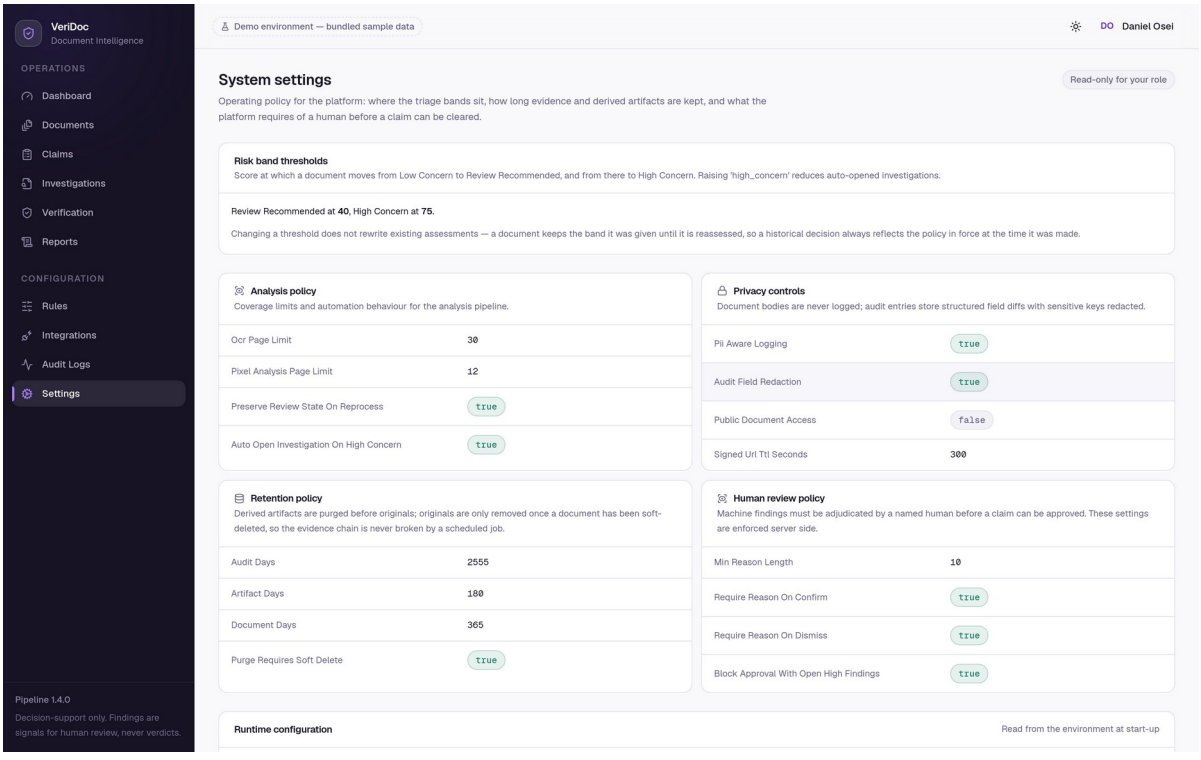


Figure 11. System settings. Band thresholds, retention, review policy and privacy policy are configuration held in the database, editable by an administrator.

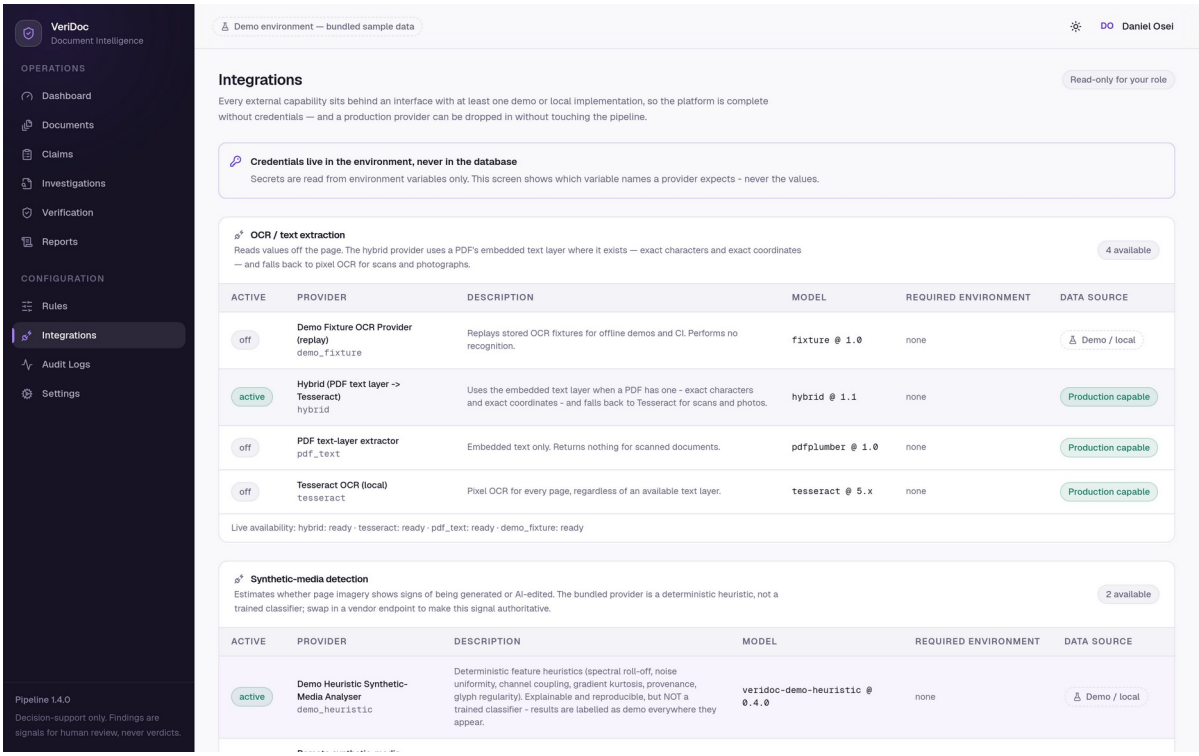


Figure 12. Provider configuration. Each provider lists the *names* of the environment variables it requires; credential values are never stored in or displayed by the platform.

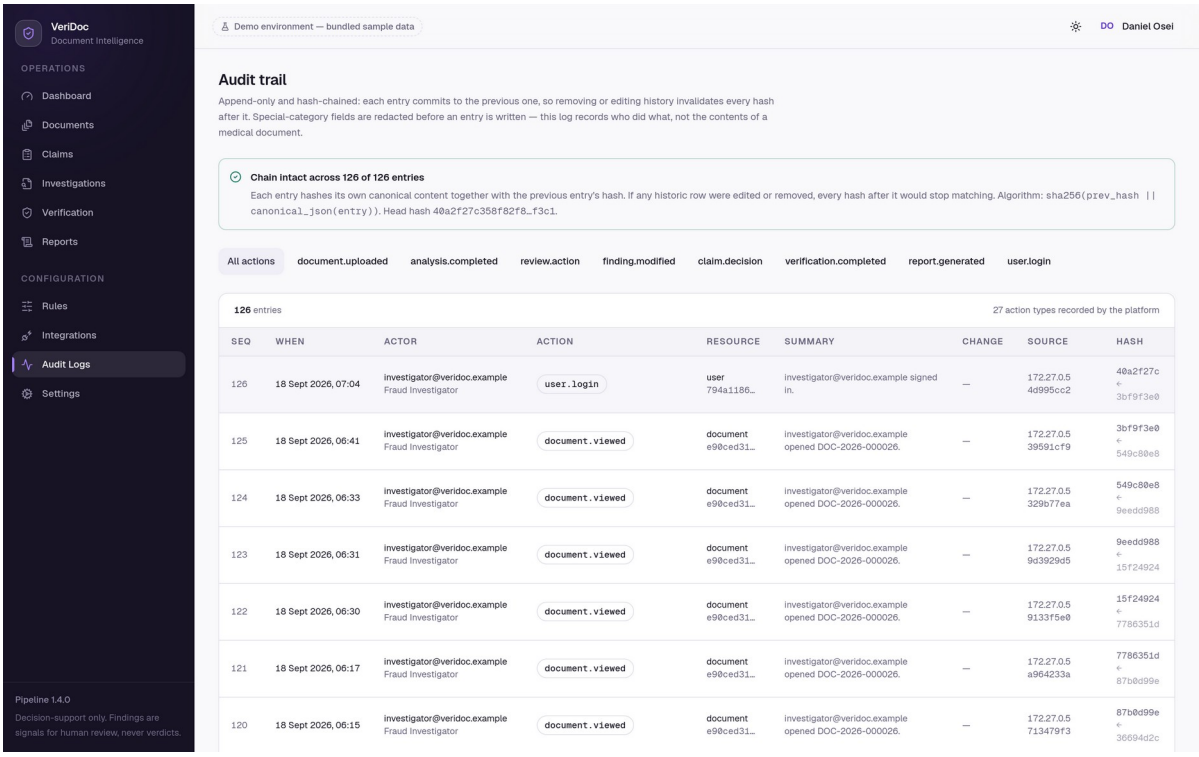


Figure 13. The audit trail with the hash chain verified at the top of the page, showing the entry and previous hashes and the before-and-after field differences.

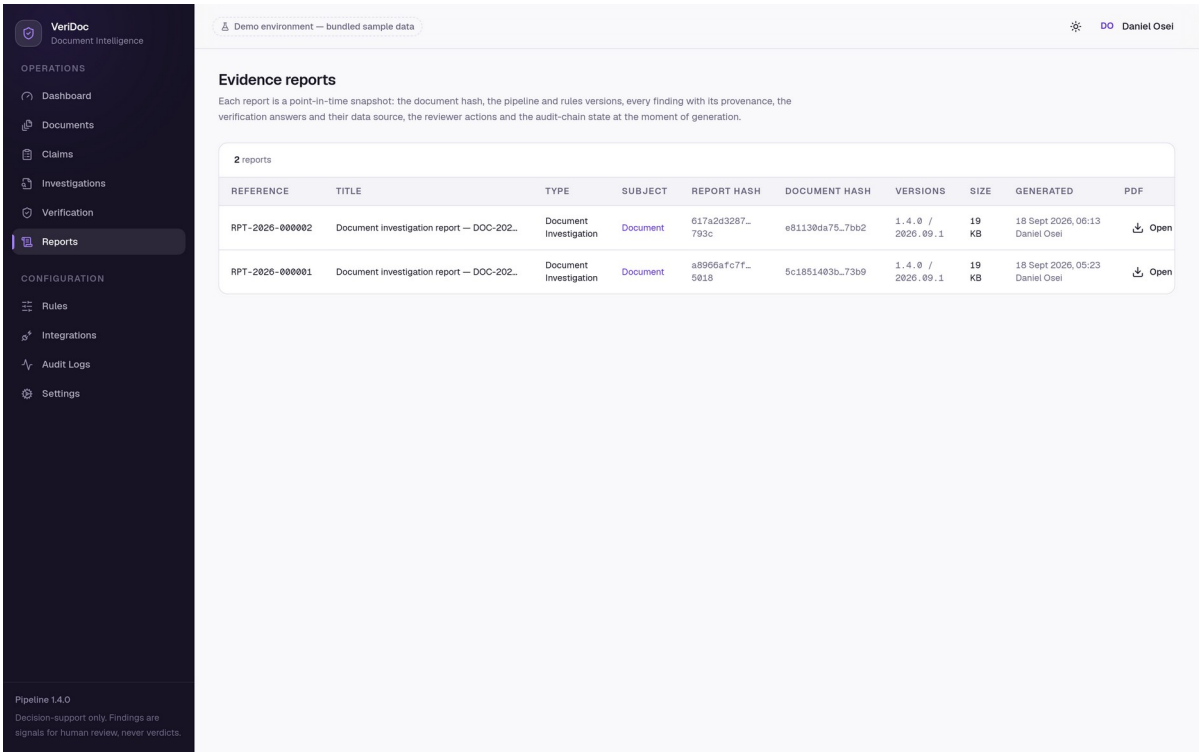


Figure 14. The report register. Each report records its own SHA-256 and the SHA-256 of the document it describes, together with the pipeline and rules versions in force.